```
//@version=6
indicator(
    title="🧬⚖️ EVA Ai+ Chart Patterns v2.9.2 · Русские метки",
    shorttitle="EVA PTRN",
    overlay=true,
    max_lines_count=500,
    max_labels_count=250,
    max_bars_back=3000)

//
------------------------------------------------------------------------------------------------------------------------
// EVA Ai+ Chart Patterns v2.9.2 · RUSSIAN LABELS
//
//
// Локализация v2.9.2:
// • все метки на графике переведены на русский язык;
// • LONG/SHORT отображаются как ЛОНГ/ШОРТ;
// • FLAG/PENNANT и разворотные модели отображаются по-русски;
// • Q заменено на КАЧ.; MACRO/MICRO — на КРУПНАЯ/ЛОКАЛЬНАЯ;
// • расчёты, геометрия, фильтры, цвета и алерты не изменены.
//
// Compile fix v2.9.1:
// • f_reversalRank() no longer wraps a binary + expression at function-body indentation;
// • the rank is calculated in an explicitly typed local float and returned on the next line;
// • reversal quality, macro bonus, pivot combinations and all pattern logic are unchanged.
//
//
// Compile fix v2.2:
// • removed the series-int loop argument from f_evaluate();
// • f_evaluate() now accepts only simple int;
// • windows 12, 16, 20 ... 80 are called directly;
// • min/max settings filter this fixed candidate grid;
// • pattern geometry, quality, drawing and breakout logic are unchanged.
//
//
// Coverage fix v2.3:
// • сохраняются два лучших независимых паттерна вместо одного;
// • длинный вымпел не вытесняется коротким флагом;
// • добавлен отдельный лимит ширины для PENNANT;
// • добавлена проверка сжатия первой и второй половины;
// • близкие окна одной фигуры не дублируются;
// • подтверждённый сигнал по-прежнему формируется только по закрытому пробою.
//
//
//
// Compile fix v2.4.1:
// • invalid type keyword line_style replaced with Pine type string;
// • line.style_dashed / line.style_dotted selection is unchanged;
// • pattern detection and drawing logic are unchanged.
//
//
//
//
//
// Pivot-combination quality fix v2.9:
// • macro reversals are selected from the best combination inside the last 10 pivots;
// • intermediate noise pivots no longer force the detector to use only the last 3/5 points;
// • Double Top/Bottom and H&S receive an independent 0–100 quality score;
// • macro candidates compete with micro candidates through one arbitration score;
```

```
// • adaptive candidate lifetime scales with the actual pattern span;
// • low-quality forming and confirmed reversal patterns are filtered separately;
// • no future data, no request.security, no historical signal backfill.
//
// Adaptive macro fix v2.8:
// • macro shoulders use relative head-height tolerance instead of ATR-only equality;
// • macro Double Top/Bottom use depth-relative equality tolerance;
// • head dominance is checked as a share of the whole macro pattern;
// • a macro pattern can confirm on the pivot-confirmation bar when its neckline
//   was already broken during the unavoidable right-pivot confirmation delay;
// • delayed confirmation is fixed on the current confirmed bar: no historical backfill;
// • micro reversal rules and continuation patterns remain unchanged.
//
// Macro reversal fix v2.7:
// • added a second coarse pivot stream for large reversal figures;
// • large Double Top/Bottom and H&S are no longer drowned by micro pivots;
// • macro scan overwrites weaker micro reversal candidates of the same family;
// • macro patterns require a wider minimum span and use their own confirmed pivots;
// • continuation logic and premium colors stay unchanged;
//
// Accuracy and visual fix v2.6:
// • every pattern family has its own color;
// • continuation quality defaults raised to suppress weak micro-patterns;
// • reversal patterns require time symmetry and prior directional context;
// • neckline confirmation requires a real closed-bar crossover/crossunder;
// • breakout candle requires a minimum directional body;
// • confirmed history is compacted to reduce chart clutter;
// • no lookahead, no request.security, no strategy calls.
//
// Reversal visibility fix v2.5:
// • reversal geometry is drawn immediately after confirmed pivot structure;
// • forming patterns use translucent thick lines;
// • confirmed neckline breakout fixes the same lines at full opacity;
// • invalidated or expired candidates are removed;
// • balanced 15m defaults improve coverage without future data;
// • FLAG/PENNANT logic is unchanged.
//
// Reversal extension v2.4:
// • Double Bottom / Double Top — thick dashed geometry;
// • Head & Shoulders / Inverse Head & Shoulders — thick dotted geometry;
// • pivot sequence is confirmed before pattern evaluation;
// • LONG/SHORT is confirmed only after a closed neckline breakout;
// • existing FLAG/PENNANT detector is not modified.
//
// Исправление v1:
// • фигура рисуется во время формирования, а не только после пробоя;
// • длина паттерна подбирается динамически в диапазоне 12–80 баров;
// • распознаются длинные вымпелы, подобные отмеченному на графике SPBE;
// • формирующаяся геометрия может обновляться вместе с новыми свечами;
// • подтверждённый LONG/SHORT фиксируется только после закрытого пробоя.
//
// ——————————————————————————————————————————————————————————————————————————————
//
// ——————————————————————————————————————————————————————————————————————————————
// 01. Поиск фигуры
```

```
//
——————————————————————————————————————————————————————————————————————————————
——————————————————————————————————————————————————————————————————————————————

string gSearch = "01 · Динамический поиск"

bool showDeveloping = input.bool(
    true,
    "Показывать формирующуюся фигуру",
    group=gSearch,
    display=display.none)

bool showFlags = input.bool(
    true,
    "Искать флаги",
    group=gSearch,
    display=display.none)

bool showPennants = input.bool(
    true,
    "Искать вымпелы",
    group=gSearch,
    display=display.none)

int minPatternLen = input.int(
    12,
    "Минимальная длина фигуры",
    minval=6,
    maxval=60,
    group=gSearch,
    display=display.none)

int maxPatternLen = input.int(
    80,
    "Максимальная длина фигуры",
    minval=16,
    maxval=80,
    group=gSearch,
    display=display.none)

// Compile-safe candidate grid: 12, 16, 20 ... 80.
// Direct constant calls are required because ta.linreg() cannot accept
// the series-int counter of a dynamic for-loop as its length/offset.
int scanStep = 4

int poleLen = input.int(
    14,
    "Длина древка",
    minval=4,
    maxval=60,
    group=gSearch,
    display=display.none)

int atrLen = input.int(
    14,
    "ATR",
    minval=2,
    maxval=100,
    group=gSearch,
    display=display.none)
```

```
float minPoleAtr = input.float(
    1.0,
    "Минимальная сила древка, ATR",
    minval=0.25,
    maxval=10.0,
    step=0.10,
    group=gSearch,
    display=display.none)

float maxPatternPoleRatio = input.float(
    1.35,
    "Макс. диапазон FLAG / древко",
    minval=0.20,
    maxval=3.00,
    step=0.05,
    group=gSearch,
    tooltip="Строгий лимит сохраняется отдельно для флагов.",
    display=display.none)

float maxPennantPoleRatio = input.float(
    2.40,
    "Макс. диапазон PENNANT / древко",
    minval=0.50,
    maxval=5.00,
    step=0.05,
    group=gSearch,
    tooltip="Распознаёт крупные вымпелы с широкой первой волной, не ослабляя FLAG.",
    display=display.none)

bool showSecondPattern = input.bool(
    true,
    "Показывать второй независимый паттерн",
    group=gSearch,
    tooltip="Одновременно показывает крупную и локальную фигуру, если они
различаются.",
    display=display.none)

int macroPatternLen = input.int(
    36,
    "Граница крупной фигуры, баров",
    minval=20,
    maxval=72,
    group=gSearch,
    display=display.none)

float pennantPriorityBonus = input.float(
    12.0,
    "Приоритет PENNANT",
    minval=0.0,
    maxval=30.0,
    step=1.0,
    group=gSearch,
    display=display.none)

float macroPriorityBonus = input.float(
    8.0,
    "Приоритет крупной фигуры",
    minval=0.0,
    maxval=30.0,
```

```
    step=1.0,
    group=gSearch,
    display=display.none)

//
——————————————————————————————————————————————————————————————————————————————
// 02. Геометрия
//
——————————————————————————————————————————————————————————————————————————————

string gGeometry = "02 · Геометрия"

float minConvergingSlope = input.float(
    0.010,
    "Мин. схождение вымпела, ATR/бар",
    minval=0.001,
    maxval=0.50,
    step=0.001,
    group=gGeometry,
    display=display.none)

float maxPennantEndRatio = input.float(
    0.78,
    "Макс. конечная ширина вымпела",
    minval=0.10,
    maxval=0.95,
    step=0.01,
    group=gGeometry,
    display=display.none)

float maxParallelDifference = input.float(
    0.10,
    "Макс. разница наклонов флага",
    minval=0.005,
    maxval=0.50,
    step=0.005,
    group=gGeometry,
    display=display.none)

float maxFlagSlope = input.float(
    0.18,
    "Макс. наклон флага, ATR/бар",
    minval=0.01,
    maxval=1.00,
    step=0.01,
    group=gGeometry,
    display=display.none)

float minEndWidthAtr = input.float(
    0.05,
    "Мин. ширина у текущей цены, ATR",
    minval=0.01,
    maxval=3.00,
    step=0.01,
    group=gGeometry,
    display=display.none)

float breakoutBufferAtr = input.float(
```

```
    0.05,
    "Буфер подтверждения пробоя, ATR",
    minval=0.00,
    maxval=1.00,
    step=0.01,
    group=gGeometry,
    display=display.none)

//
─────────────────────────────────────────────────────────────────────────────────────────────────────────────
// 03. Качество и объём
//
─────────────────────────────────────────────────────────────────────────────────────────────────────────────

string gQuality = "03 · Качество"

float minDevelopingQuality = input.float(
    55.0,
    "Мин. качество формирующейся фигуры",
    minval=1.0,
    maxval=100.0,
    step=1.0,
    group=gQuality,
    display=display.none)

float minConfirmedQuality = input.float(
    65.0,
    "Мин. качество подтверждённого сигнала",
    minval=1.0,
    maxval=100.0,
    step=1.0,
    group=gQuality,
    display=display.none)

bool useVolumeContraction = input.bool(
    false,
    "Жёстко требовать снижение объёма",
    group=gQuality,
    tooltip="По умолчанию объём участвует в оценке, но не блокирует отрисовку
геометрически правильной фигуры.",
    display=display.none)

float maxConsolidationVolumeRatio = input.float(
    1.05,
    "Макс. объём фигуры / объём древка",
    minval=0.10,
    maxval=3.00,
    step=0.05,
    group=gQuality,
    display=display.none)

bool useBreakoutVolume = input.bool(
    false,
    "Жёстко требовать объём пробоя",
    group=gQuality,
    display=display.none)

int breakoutVolumeLen = input.int(
```

```
    20,
    "Средний объём пробоя",
    minval=2,
    maxval=100,
    group=gQuality,
    display=display.none)

float minBreakoutVolumeRatio = input.float(
    1.05,
    "Мин. объём пробоя / средний",
    minval=0.10,
    maxval=5.00,
    step=0.05,
    group=gQuality,
    display=display.none)

int signalCooldown = input.int(
    12,
    "Пауза между сигналами",
    minval=0,
    maxval=100,
    group=gQuality,
    display=display.none)

//
——————————————————————————————————————————————————————————————————————————————————————————————————————————
// 04. Визуал EVA
//
——————————————————————————————————————————————————————————————————————————————————————————————————————————

string gVisual = "04 · Визуал EVA"

int projectionBars = input.int(
    18,
    "Продление границ",
    minval=2,
    maxval=100,
    group=gVisual,
    display=display.none)

int maxApexProjection = input.int(
    40,
    "Макс. проекция вершины вымпела",
    minval=5,
    maxval=200,
    group=gVisual,
    display=display.none)

bool showTarget = input.bool(
    true,
    "Показывать цель после пробоя",
    group=gVisual,
    display=display.none)

float targetFactor = input.float(
    1.0,
    "Цель × длина древка",
    minval=0.25,
```

```
    maxval=3.0,
    step=0.25,
    group=gVisual,
    display=display.none)

int targetBars = input.int(
    25,
    "Длина линии цели",
    minval=3,
    maxval=200,
    group=gVisual,
    display=display.none)

int maxConfirmedPatterns = input.int(
    12,
    "Максимум подтверждённых фигур",
    minval=1,
    maxval=50,
    group=gVisual,
    display=display.none)

color longColor = input.color(
    color.rgb(0, 200, 150),
    "ФЛАГ ЛОНГ",
    group=gVisual,
    display=display.none)

color shortColor = input.color(
    color.rgb(255, 77, 109),
    "ФЛАГ ШОРТ",
    group=gVisual,
    display=display.none)

color developingColor = input.color(
    color.rgb(219, 180, 84),
    "Формирующаяся фигура",
    group=gVisual,
    display=display.none)

color targetColor = input.color(
    color.rgb(104, 181, 255),
    "Цель",
    group=gVisual,
    display=display.none)


color pennantLongColor = input.color(
    color.rgb(0, 184, 217),
    "ВЫМПЕЛ ЛОНГ",
    group=gVisual,
    display=display.none)

color pennantShortColor = input.color(
    color.rgb(255, 159, 28),
    "ВЫМПЕЛ ШОРТ",
    group=gVisual,
    display=display.none)

color doubleBottomColor = input.color(
    color.rgb(143, 214, 55),
```

```
    "ДВОЙНОЕ ДНО",
    group=gVisual,
    display=display.none)

color doubleTopColor = input.color(
    color.rgb(255, 79, 163),
    "ДВОЙНАЯ ВЕРШИНА",
    group=gVisual,
    display=display.none)

color headShouldersColor = input.color(
    color.rgb(168, 85, 247),
    "ГОЛОВА И ПЛЕЧИ",
    group=gVisual,
    display=display.none)

color inverseHeadShouldersColor = input.color(
    color.rgb(77, 163, 255),
    "ПЕРЕВЁРНУТЫЕ ГОЛОВА И ПЛЕЧИ",
    group=gVisual,
    display=display.none)

int formingPatternTransparency = input.int(
    58,
    "Прозрачность формирующейся фигуры",
    minval=20,
    maxval=90,
    group=gVisual,
    display=display.none)


//
------------------------------------------------------------------------------------------------------------------------
// 05. Разворотные фигуры
//
------------------------------------------------------------------------------------------------------------------------

string gReversal = "05 · Разворотные фигуры"

bool showDoublePatterns = input.bool(
    true,
    "Двойное дно / двойная вершина",
    group=gReversal,
    display=display.none)

bool showHeadShoulders = input.bool(
    true,
    "Голова и плечи / перевёрнутая",
    group=gReversal,
    display=display.none)

bool showFormingReversals = input.bool(
    true,
    "Показывать формирующиеся разворотные фигуры",
    group=gReversal,
    tooltip="После подтверждения pivot-точек фигура отображается полупрозрачно. После
пробоя neckline фиксируется ярко.",
    display=display.none)
```

```
int reversalPivotLeft = input.int(
    3,
    "Pivot: свечей слева",
    minval=2,
    maxval=20,
    group=gReversal,
    display=display.none)

int reversalPivotRight = input.int(
    3,
    "Pivot: свечей справа",
    minval=2,
    maxval=20,
    group=gReversal,
    tooltip="Фигура строится только после подтверждения экстремума справа.",
    display=display.none)

int minPivotGap = input.int(
    2,
    "Минимум баров между точками",
    minval=1,
    maxval=30,
    group=gReversal,
    display=display.none)

int minReversalSpan = input.int(
    8,
    "Минимальная длина фигуры",
    minval=5,
    maxval=100,
    group=gReversal,
    display=display.none)

int maxReversalSpan = input.int(
    140,
    "Максимальная длина фигуры",
    minval=20,
    maxval=500,
    group=gReversal,
    display=display.none)

int reversalConfirmBars = input.int(
    60,
    "Ожидание пробоя neckline, баров",
    minval=5,
    maxval=300,
    group=gReversal,
    display=display.none)

float doubleToleranceAtr = input.float(
    0.45,
    "Допуск двух вершин/донов, ATR",
    minval=0.05,
    maxval=2.00,
    step=0.05,
    group=gReversal,
    display=display.none)

float minDoubleDepthAtr = input.float(
```

```
    0.75,
    "Мин. глубина двойной фигуры, ATR",
    minval=0.10,
    maxval=5.00,
    step=0.10,
    group=gReversal,
    display=display.none)

float shoulderToleranceAtr = input.float(
    0.60,
    "Допуск плеч, ATR",
    minval=0.05,
    maxval=3.00,
    step=0.05,
    group=gReversal,
    display=display.none)

float minHeadHeightAtr = input.float(
    0.70,
    "Мин. высота головы, ATR",
    minval=0.10,
    maxval=5.00,
    step=0.05,
    group=gReversal,
    display=display.none)

float maxNecklineSlopeAtr = input.float(
    0.20,
    "Макс. наклон neckline, ATR/бар",
    minval=0.01,
    maxval=1.00,
    step=0.01,
    group=gReversal,
    display=display.none)


bool requireReversalTrendContext = input.bool(
    true,
    "Требовать предшествующее направленное движение",
    group=gReversal,
    display=display.none)

float minApproachMoveAtr = input.float(
    0.75,
    "Мин. движение до фигуры, ATR",
    minval=0.10,
    maxval=5.00,
    step=0.05,
    group=gReversal,
    display=display.none)

float maxDoubleTimeRatio = input.float(
    2.20,
    "Макс. асимметрия Double Top/Bottom",
    minval=1.00,
    maxval=5.00,
    step=0.10,
    group=gReversal,
    tooltip="Отношение длин левой и правой половины фигуры.",
    display=display.none)
```

```
float maxShoulderTimeRatio = input.float(
    2.00,
    "Макс. асимметрия плеч",
    minval=1.00,
    maxval=5.00,
    step=0.10,
    group=gReversal,
    display=display.none)

float minShoulderDepthAtr = input.float(
    0.35,
    "Мин. глубина плеч относительно neckline, ATR",
    minval=0.05,
    maxval=3.00,
    step=0.05,
    group=gReversal,
    display=display.none)

float reversalBreakoutBufferAtr = input.float(
    0.05,
    "Буфер пробоя neckline, ATR",
    minval=0.00,
    maxval=1.00,
    step=0.01,
    group=gReversal,
    display=display.none)

float minReversalBreakoutBodyAtr = input.float(
    0.18,
    "Мин. тело свечи пробоя, ATR",
    minval=0.00,
    maxval=2.00,
    step=0.01,
    group=gReversal,
    display=display.none)

bool enableMacroReversals = input.bool(
    true,
    "Искать крупные разворотные фигуры",
    group=gReversal,
    tooltip="Дополнительный грубый pivot-поток для крупных Double Top/Bottom и Head &
Shoulders.",
    display=display.none)

int macroReversalPivotLeft = input.int(
    7,
    "Macro Pivot: свечей слева",
    minval=3,
    maxval=30,
    group=gReversal,
    display=display.none)

int macroReversalPivotRight = input.int(
    7,
    "Macro Pivot: свечей справа",
    minval=3,
    maxval=30,
    group=gReversal,
    display=display.none)
```

```
int macroMinReversalSpan = input.int(
    18,
    "Минимальная длина крупной фигуры",
    minval=8,
    maxval=200,
    group=gReversal,
    display=display.none)


float macroShoulderToleranceRatio = input.float(
    0.45,
    "Macro H&S: допуск плеч / высота головы",
    minval=0.10,
    maxval=0.90,
    step=0.05,
    group=gReversal,
    tooltip="Для крупных фигур допуск плеч считается адаптивно: максимум из ATR-допуска
и доли высоты головы.",
    display=display.none)

float macroDoubleToleranceRatio = input.float(
    0.30,
    "Macro Double: допуск экстремумов / глубина",
    minval=0.05,
    maxval=0.80,
    step=0.05,
    group=gReversal,
    tooltip="Для крупных двойных фигур сравнение экстремумов масштабируется
относительно глубины модели.",
    display=display.none)

float macroMinHeadDominanceRatio = input.float(
    0.22,
    "Macro H&S: мин. доминирование головы",
    minval=0.05,
    maxval=0.70,
    step=0.01,
    group=gReversal,
    tooltip="Доля высоты всей модели, на которую голова должна выступать над плечами.",
    display=display.none)

bool macroAllowDelayedBreakout = input.bool(
    true,
    "Подтверждать Macro после задержки Pivot",
    group=gReversal,
    tooltip="Если neckline уже пробит к моменту подтверждения правого Macro Pivot, фигура
фиксируется на текущей закрытой свече без дорисовки сигнала в прошлое.",
    display=display.none)

float macroLateConfirmBufferAtr = input.float(
    0.12,
    "Macro: буфер позднего подтверждения, ATR",
    minval=0.02,
    maxval=1.00,
    step=0.01,
    group=gReversal,
    display=display.none)
```

```
bool enableMacroCombinationSearch = input.bool(
    true,
    "Macro: поиск лучшей комбинации Pivot",
    group=gReversal,
    tooltip="Перебирает допустимые комбинации среди последних Macro Pivot и выбирает
наиболее качественную крупную фигуру.",
    display=display.none)

int macroCombinationLookback = input.int(
    10,
    "Macro: Pivot в окне поиска",
    minval=6,
    maxval=12,
    group=gReversal,
    display=display.none)

int macroMaxTailPivots = input.int(
    2,
    "Macro: допустимые Pivot после фигуры",
    minval=0,
    maxval=4,
    group=gReversal,
    tooltip="Позволяет сохранить крупную фигуру, если после правого плеча или второй
вершины появился небольшой шумовой Pivot.",
    display=display.none)

float minFormingReversalQuality = input.float(
    58.0,
    "Мин. Q формирующейся разворотной фигуры",
    minval=1.0,
    maxval=100.0,
    step=1.0,
    group=gReversal,
    display=display.none)

float minConfirmedReversalQuality = input.float(
    68.0,
    "Мин. Q подтверждённой разворотной фигуры",
    minval=1.0,
    maxval=100.0,
    step=1.0,
    group=gReversal,
    display=display.none)

float reversalReplaceMargin = input.float(
    3.0,
    "Минимальный перевес нового кандидата Q",
    minval=0.0,
    maxval=25.0,
    step=1.0,
    group=gReversal,
    display=display.none)

float macroArbitrationBonus = input.float(
    6.0,
    "Приоритет Macro-кандидата",
    minval=0.0,
    maxval=25.0,
    step=1.0,
    group=gReversal,
```

```
    tooltip="Используется только при выборе между пересекающимися Micro и Macro
кандидатами. В отображаемый Q не добавляется.",
    display=display.none)

bool adaptiveReversalLifetime = input.bool(
    true,
    "Адаптивный срок жизни разворотной фигуры",
    group=gReversal,
    display=display.none)

float microReversalLifetimeFactor = input.float(
    1.0,
    "Micro: срок ожидания × длина фигуры",
    minval=0.25,
    maxval=4.0,
    step=0.05,
    group=gReversal,
    display=display.none)

float macroReversalLifetimeFactor = input.float(
    1.45,
    "Macro: срок ожидания × длина фигуры",
    minval=0.25,
    maxval=4.0,
    step=0.05,
    group=gReversal,
    display=display.none)

int minAdaptiveReversalLifetime = input.int(
    12,
    "Мин. срок ожидания пробоя",
    minval=3,
    maxval=100,
    group=gReversal,
    display=display.none)

int maxAdaptiveReversalLifetime = input.int(
    220,
    "Макс. срок ожидания пробоя",
    minval=20,
    maxval=500,
    group=gReversal,
    display=display.none)

//
------------------------------------------------------------------------------
// Хранилище подтверждённых объектов
//
------------------------------------------------------------------------------

var array<line> confirmedLines = array.new<line>()
var array<label> confirmedLabels = array.new<label>()

f_keepLine(line id) =>
    array.push(confirmedLines, id)
    true

f_keepLabel(label id) =>
```

```
    array.push(confirmedLabels, id)
    true

f_trimConfirmed() =>
    int maxLines = maxConfirmedPatterns * 8

    while array.size(confirmedLines) > maxLines
        line oldLine = array.shift(confirmedLines)
        line.delete(oldLine)

    while array.size(confirmedLabels) > maxConfirmedPatterns
        label oldLabel = array.shift(confirmedLabels)
        label.delete(oldLabel)

    true

//
------------------------------------------------------------------------------------------------------------------------
// Оценка одного окна
//
// kind:
// 0 — нет фигуры
// 1 — FLAG
// 2 — PENNANT
//
// bias:
// +1 — продолжение LONG
// -1 — продолжение SHORT
//
------------------------------------------------------------------------------------------------------------------------

float atr = ta.atr(atrLen)

f_evaluate(simple int patternLen) =>
    float localAtr = atr[patternLen]
    float upperStart = ta.linreg(high[1], patternLen, patternLen - 1)
    float upperEnd = ta.linreg(high[1], patternLen, 0)
    float lowerStart = ta.linreg(low[1], patternLen, patternLen - 1)
    float lowerEnd = ta.linreg(low[1], patternLen, 0)

    float divisor = math.max(patternLen - 1, 1)
    float upperSlope = (upperEnd - upperStart) / divisor
    float lowerSlope = (lowerEnd - lowerStart) / divisor

    float upperSlopeAtr =
        not na(localAtr) and localAtr > 0.0 ?
        upperSlope / localAtr :
        na

    float lowerSlopeAtr =
        not na(localAtr) and localAtr > 0.0 ?
        lowerSlope / localAtr :
        na

    float widthStart = upperStart - lowerStart
    float widthEnd = upperEnd - lowerEnd
    float widthEndAtr =
        not na(localAtr) and localAtr > 0.0 ?
```

```
    widthEnd / localAtr :
    0.0

float patternHigh = ta.highest(high[1], patternLen)
float patternLow = ta.lowest(low[1], patternLen)
float patternRange = patternHigh - patternLow

float poleStart = close[patternLen + poleLen]
float poleEnd = close[patternLen]
float poleMove = poleEnd - poleStart
float poleSize = math.abs(poleMove)

float poleStrength =
    not na(localAtr) and localAtr > 0.0 ?
    poleSize / localAtr :
    0.0

int bias = poleMove > 0.0 ? 1 : poleMove < 0.0 ? -1 : 0

float patternPoleRatio =
    poleSize > syminfo.mintick ?
    patternRange / poleSize :
    100.0

float endRatio =
    widthStart > syminfo.mintick ?
    widthEnd / widthStart :
    10.0

int halfLen = math.max(int(patternLen / 2), 3)

float earlyHigh = ta.highest(high[halfLen + 1], halfLen)
float earlyLow = ta.lowest(low[halfLen + 1], halfLen)
float lateHigh = ta.highest(high[1], halfLen)
float lateLow = ta.lowest(low[1], halfLen)

float earlyWidth = earlyHigh - earlyLow
float lateWidth = lateHigh - lateLow

bool regressionPennant =
    upperSlopeAtr <= -minConvergingSlope and
    lowerSlopeAtr >= minConvergingSlope and
    widthStart > 0.0 and
    widthEnd > 0.0 and
    endRatio <= maxPennantEndRatio

bool envelopePennant =
    not na(earlyWidth) and
    not na(lateWidth) and
    earlyWidth > syminfo.mintick and
    lateWidth > syminfo.mintick and
    lateHigh < earlyHigh and
    lateLow > earlyLow and
    lateWidth <= earlyWidth * 0.88

bool pennantShape =
    showPennants and
    (regressionPennant or envelopePennant)

bool longFlagShape =
```

```
    showFlags and
    bias == 1 and
    upperSlopeAtr >= -maxFlagSlope and
    upperSlopeAtr <= 0.05 and
    lowerSlopeAtr >= -maxFlagSlope and
    lowerSlopeAtr <= 0.05 and
    math.abs(upperSlopeAtr - lowerSlopeAtr) <= maxParallelDifference

bool shortFlagShape =
    showFlags and
    bias == -1 and
    upperSlopeAtr >= -0.05 and
    upperSlopeAtr <= maxFlagSlope and
    lowerSlopeAtr >= -0.05 and
    lowerSlopeAtr <= maxFlagSlope and
    math.abs(upperSlopeAtr - lowerSlopeAtr) <= maxParallelDifference

int kind =
    pennantShape ? 2 :
    longFlagShape or shortFlagShape ? 1 :
    0

float poleVolume = ta.sma(volume[patternLen + 1], poleLen)
float patternVolume = ta.sma(volume[1], patternLen)

float volumeRatio =
    not na(poleVolume) and poleVolume > 0.0 ?
    patternVolume / poleVolume :
    1.0

bool volumeOk =
    not useVolumeContraction or
    volumeRatio <= maxConsolidationVolumeRatio

float allowedPatternPoleRatio =
    kind == 2 ?
    maxPennantPoleRatio :
    maxPatternPoleRatio

bool baseValid =
    bar_index >= patternLen + poleLen + breakoutVolumeLen + 5 and
    not na(localAtr) and
    localAtr > 0.0 and
    bias != 0 and
    kind != 0 and
    poleStrength >= minPoleAtr and
    patternPoleRatio <= allowedPatternPoleRatio and
    widthEndAtr >= minEndWidthAtr and
    volumeOk

float compression =
    widthStart > syminfo.mintick ?
    math.max(0.0, math.min(1.0, 1.0 - endRatio)) :
    0.0

float poleScore =
    math.min(
        30.0,
        poleStrength /
        math.max(minPoleAtr, 0.01) * 20.0)
```

```
    float geometryScore =
        kind == 2 ?
        math.min(30.0, compression * 45.0) :
        math.max(
            0.0,
            math.min(
                30.0,
                (maxParallelDifference -
                 math.abs(upperSlopeAtr - lowerSlopeAtr)) /
                math.max(maxParallelDifference, 0.001) * 30.0))

    float compactScore =
        math.max(
            0.0,
            math.min(
                20.0,
                (allowedPatternPoleRatio - patternPoleRatio) /
                math.max(allowedPatternPoleRatio, 0.01) * 20.0))

    float spanScore =
        math.min(
            10.0,
            patternLen /
            math.max(maxPatternLen, 1) * 10.0)

    float volumeScore =
        math.max(
            0.0,
            math.min(
                10.0,
                (1.30 - volumeRatio) * 20.0))

    float quality =
        math.min(
            100.0,
            poleScore +
            geometryScore +
            compactScore +
            spanScore +
            volumeScore)

    [baseValid,
     quality,
     kind,
     bias,
     upperStart,
     upperEnd,
     lowerStart,
     lowerEnd,
     upperSlope,
     lowerSlope,
     poleStart,
     poleEnd,
     poleSize,
     volumeRatio]

//
_____
_____
```

```
// Динамический выбор лучшего окна
//
_____________________________________________________________________________
_____________________________________________________________

int effectiveMinLen = math.min(minPatternLen, maxPatternLen)
int effectiveMaxLen = math.max(minPatternLen, maxPatternLen)

bool bestValid = false
float bestQuality = -1.0
float bestRank = -1.0
int bestKind = 0
int bestBias = 0
int bestLen = na

float bestUpperStart = na
float bestUpperEnd = na
float bestLowerStart = na
float bestLowerEnd = na
float bestUpperSlope = na
float bestLowerSlope = na
float bestPoleStart = na
float bestPoleEnd = na
float bestPoleSize = na
float bestVolumeRatio = na

bool secondValid = false
float secondQuality = -1.0
float secondRank = -1.0
int secondKind = 0
int secondBias = 0
int secondLen = na

float secondUpperStart = na
float secondUpperEnd = na
float secondLowerStart = na
float secondLowerEnd = na
float secondUpperSlope = na
float secondLowerSlope = na
float secondPoleStart = na
float secondPoleEnd = na
float secondPoleSize = na
float secondVolumeRatio = na

[c12Valid,
 c12Quality,
 c12Kind,
 c12Bias,
 c12UpperStart,
 c12UpperEnd,
 c12LowerStart,
 c12LowerEnd,
 c12UpperSlope,
 c12LowerSlope,
 c12PoleStart,
 c12PoleEnd,
 c12PoleSize,
 c12VolumeRatio] = f_evaluate(12)

bool c12Allowed =
```

```
    12 >= effectiveMinLen and
    12 <= effectiveMaxLen

float c12Rank =
    c12Quality +
    (c12Kind == 2 ? pennantPriorityBonus : 0.0) +
    (12 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c12Allowed and c12Valid
   if c12Rank > bestRank
      secondValid := bestValid
      secondQuality := bestQuality
      secondRank := bestRank
      secondKind := bestKind
      secondBias := bestBias
      secondLen := bestLen
      secondUpperStart := bestUpperStart
      secondUpperEnd := bestUpperEnd
      secondLowerStart := bestLowerStart
      secondLowerEnd := bestLowerEnd
      secondUpperSlope := bestUpperSlope
      secondLowerSlope := bestLowerSlope
      secondPoleStart := bestPoleStart
      secondPoleEnd := bestPoleEnd
      secondPoleSize := bestPoleSize
      secondVolumeRatio := bestVolumeRatio

      bestValid := true
      bestQuality := c12Quality
      bestRank := c12Rank
      bestKind := c12Kind
      bestBias := c12Bias
      bestLen := 12
      bestUpperStart := c12UpperStart
      bestUpperEnd := c12UpperEnd
      bestLowerStart := c12LowerStart
      bestLowerEnd := c12LowerEnd
      bestUpperSlope := c12UpperSlope
      bestLowerSlope := c12LowerSlope
      bestPoleStart := c12PoleStart
      bestPoleEnd := c12PoleEnd
      bestPoleSize := c12PoleSize
      bestVolumeRatio := c12VolumeRatio
   else
      bool c12Distinct =
          na(bestLen) or
          math.abs(12 - bestLen) >= 8 or
          c12Kind != bestKind

      if c12Distinct and c12Rank > secondRank
         secondValid := true
         secondQuality := c12Quality
         secondRank := c12Rank
         secondKind := c12Kind
         secondBias := c12Bias
         secondLen := 12
         secondUpperStart := c12UpperStart
         secondUpperEnd := c12UpperEnd
         secondLowerStart := c12LowerStart
         secondLowerEnd := c12LowerEnd
```

```
            secondUpperSlope := c12UpperSlope
            secondLowerSlope := c12LowerSlope
            secondPoleStart := c12PoleStart
            secondPoleEnd := c12PoleEnd
            secondPoleSize := c12PoleSize
            secondVolumeRatio := c12VolumeRatio

[c16Valid,
 c16Quality,
 c16Kind,
 c16Bias,
 c16UpperStart,
 c16UpperEnd,
 c16LowerStart,
 c16LowerEnd,
 c16UpperSlope,
 c16LowerSlope,
 c16PoleStart,
 c16PoleEnd,
 c16PoleSize,
 c16VolumeRatio] = f_evaluate(16)

bool c16Allowed =
     16 >= effectiveMinLen and
     16 <= effectiveMaxLen

float c16Rank =
     c16Quality +
     (c16Kind == 2 ? pennantPriorityBonus : 0.0) +
     (16 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c16Allowed and c16Valid
    if c16Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c16Quality
        bestRank := c16Rank
        bestKind := c16Kind
        bestBias := c16Bias
        bestLen := 16
        bestUpperStart := c16UpperStart
        bestUpperEnd := c16UpperEnd
        bestLowerStart := c16LowerStart
        bestLowerEnd := c16LowerEnd
```

```
        bestUpperSlope := c16UpperSlope
        bestLowerSlope := c16LowerSlope
        bestPoleStart := c16PoleStart
        bestPoleEnd := c16PoleEnd
        bestPoleSize := c16PoleSize
        bestVolumeRatio := c16VolumeRatio
    else
        bool c16Distinct =
            na(bestLen) or
            math.abs(16 - bestLen) >= 8 or
            c16Kind != bestKind

        if c16Distinct and c16Rank > secondRank
            secondValid := true
            secondQuality := c16Quality
            secondRank := c16Rank
            secondKind := c16Kind
            secondBias := c16Bias
            secondLen := 16
            secondUpperStart := c16UpperStart
            secondUpperEnd := c16UpperEnd
            secondLowerStart := c16LowerStart
            secondLowerEnd := c16LowerEnd
            secondUpperSlope := c16UpperSlope
            secondLowerSlope := c16LowerSlope
            secondPoleStart := c16PoleStart
            secondPoleEnd := c16PoleEnd
            secondPoleSize := c16PoleSize
            secondVolumeRatio := c16VolumeRatio

[c20Valid,
 c20Quality,
 c20Kind,
 c20Bias,
 c20UpperStart,
 c20UpperEnd,
 c20LowerStart,
 c20LowerEnd,
 c20UpperSlope,
 c20LowerSlope,
 c20PoleStart,
 c20PoleEnd,
 c20PoleSize,
 c20VolumeRatio] = f_evaluate(20)

bool c20Allowed =
    20 >= effectiveMinLen and
    20 <= effectiveMaxLen

float c20Rank =
    c20Quality +
    (c20Kind == 2 ? pennantPriorityBonus : 0.0) +
    (20 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c20Allowed and c20Valid
    if c20Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
```

```
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c20Quality
        bestRank := c20Rank
        bestKind := c20Kind
        bestBias := c20Bias
        bestLen := 20
        bestUpperStart := c20UpperStart
        bestUpperEnd := c20UpperEnd
        bestLowerStart := c20LowerStart
        bestLowerEnd := c20LowerEnd
        bestUpperSlope := c20UpperSlope
        bestLowerSlope := c20LowerSlope
        bestPoleStart := c20PoleStart
        bestPoleEnd := c20PoleEnd
        bestPoleSize := c20PoleSize
        bestVolumeRatio := c20VolumeRatio
    else
        bool c20Distinct =
            na(bestLen) or
            math.abs(20 - bestLen) >= 8 or
            c20Kind != bestKind

        if c20Distinct and c20Rank > secondRank
            secondValid := true
            secondQuality := c20Quality
            secondRank := c20Rank
            secondKind := c20Kind
            secondBias := c20Bias
            secondLen := 20
            secondUpperStart := c20UpperStart
            secondUpperEnd := c20UpperEnd
            secondLowerStart := c20LowerStart
            secondLowerEnd := c20LowerEnd
            secondUpperSlope := c20UpperSlope
            secondLowerSlope := c20LowerSlope
            secondPoleStart := c20PoleStart
            secondPoleEnd := c20PoleEnd
            secondPoleSize := c20PoleSize
            secondVolumeRatio := c20VolumeRatio

[c24Valid,
 c24Quality,
 c24Kind,
 c24Bias,
 c24UpperStart,
 c24UpperEnd,
 c24LowerStart,
```

```
 c24LowerEnd,
 c24UpperSlope,
 c24LowerSlope,
 c24PoleStart,
 c24PoleEnd,
 c24PoleSize,
 c24VolumeRatio] = f_evaluate(24)

bool c24Allowed =
    24 >= effectiveMinLen and
    24 <= effectiveMaxLen

float c24Rank =
    c24Quality +
    (c24Kind == 2 ? pennantPriorityBonus : 0.0) +
    (24 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c24Allowed and c24Valid
   if c24Rank > bestRank
      secondValid := bestValid
      secondQuality := bestQuality
      secondRank := bestRank
      secondKind := bestKind
      secondBias := bestBias
      secondLen := bestLen
      secondUpperStart := bestUpperStart
      secondUpperEnd := bestUpperEnd
      secondLowerStart := bestLowerStart
      secondLowerEnd := bestLowerEnd
      secondUpperSlope := bestUpperSlope
      secondLowerSlope := bestLowerSlope
      secondPoleStart := bestPoleStart
      secondPoleEnd := bestPoleEnd
      secondPoleSize := bestPoleSize
      secondVolumeRatio := bestVolumeRatio

      bestValid := true
      bestQuality := c24Quality
      bestRank := c24Rank
      bestKind := c24Kind
      bestBias := c24Bias
      bestLen := 24
      bestUpperStart := c24UpperStart
      bestUpperEnd := c24UpperEnd
      bestLowerStart := c24LowerStart
      bestLowerEnd := c24LowerEnd
      bestUpperSlope := c24UpperSlope
      bestLowerSlope := c24LowerSlope
      bestPoleStart := c24PoleStart
      bestPoleEnd := c24PoleEnd
      bestPoleSize := c24PoleSize
      bestVolumeRatio := c24VolumeRatio
   else
      bool c24Distinct =
          na(bestLen) or
          math.abs(24 - bestLen) >= 8 or
          c24Kind != bestKind

      if c24Distinct and c24Rank > secondRank
         secondValid := true
```

```
            secondQuality := c24Quality
            secondRank := c24Rank
            secondKind := c24Kind
            secondBias := c24Bias
            secondLen := 24
            secondUpperStart := c24UpperStart
            secondUpperEnd := c24UpperEnd
            secondLowerStart := c24LowerStart
            secondLowerEnd := c24LowerEnd
            secondUpperSlope := c24UpperSlope
            secondLowerSlope := c24LowerSlope
            secondPoleStart := c24PoleStart
            secondPoleEnd := c24PoleEnd
            secondPoleSize := c24PoleSize
            secondVolumeRatio := c24VolumeRatio

[c28Valid,
 c28Quality,
 c28Kind,
 c28Bias,
 c28UpperStart,
 c28UpperEnd,
 c28LowerStart,
 c28LowerEnd,
 c28UpperSlope,
 c28LowerSlope,
 c28PoleStart,
 c28PoleEnd,
 c28PoleSize,
 c28VolumeRatio] = f_evaluate(28)

bool c28Allowed =
    28 >= effectiveMinLen and
    28 <= effectiveMaxLen

float c28Rank =
    c28Quality +
    (c28Kind == 2 ? pennantPriorityBonus : 0.0) +
    (28 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c28Allowed and c28Valid
   if c28Rank > bestRank
       secondValid := bestValid
       secondQuality := bestQuality
       secondRank := bestRank
       secondKind := bestKind
       secondBias := bestBias
       secondLen := bestLen
       secondUpperStart := bestUpperStart
       secondUpperEnd := bestUpperEnd
       secondLowerStart := bestLowerStart
       secondLowerEnd := bestLowerEnd
       secondUpperSlope := bestUpperSlope
       secondLowerSlope := bestLowerSlope
       secondPoleStart := bestPoleStart
       secondPoleEnd := bestPoleEnd
       secondPoleSize := bestPoleSize
       secondVolumeRatio := bestVolumeRatio

       bestValid := true
```

```
        bestQuality := c28Quality
        bestRank := c28Rank
        bestKind := c28Kind
        bestBias := c28Bias
        bestLen := 28
        bestUpperStart := c28UpperStart
        bestUpperEnd := c28UpperEnd
        bestLowerStart := c28LowerStart
        bestLowerEnd := c28LowerEnd
        bestUpperSlope := c28UpperSlope
        bestLowerSlope := c28LowerSlope
        bestPoleStart := c28PoleStart
        bestPoleEnd := c28PoleEnd
        bestPoleSize := c28PoleSize
        bestVolumeRatio := c28VolumeRatio
    else
        bool c28Distinct =
            na(bestLen) or
            math.abs(28 - bestLen) >= 8 or
            c28Kind != bestKind

        if c28Distinct and c28Rank > secondRank
            secondValid := true
            secondQuality := c28Quality
            secondRank := c28Rank
            secondKind := c28Kind
            secondBias := c28Bias
            secondLen := 28
            secondUpperStart := c28UpperStart
            secondUpperEnd := c28UpperEnd
            secondLowerStart := c28LowerStart
            secondLowerEnd := c28LowerEnd
            secondUpperSlope := c28UpperSlope
            secondLowerSlope := c28LowerSlope
            secondPoleStart := c28PoleStart
            secondPoleEnd := c28PoleEnd
            secondPoleSize := c28PoleSize
            secondVolumeRatio := c28VolumeRatio

[c32Valid,
 c32Quality,
 c32Kind,
 c32Bias,
 c32UpperStart,
 c32UpperEnd,
 c32LowerStart,
 c32LowerEnd,
 c32UpperSlope,
 c32LowerSlope,
 c32PoleStart,
 c32PoleEnd,
 c32PoleSize,
 c32VolumeRatio] = f_evaluate(32)

bool c32Allowed =
    32 >= effectiveMinLen and
    32 <= effectiveMaxLen

float c32Rank =
    c32Quality +
```

```
     (c32Kind == 2 ? pennantPriorityBonus : 0.0) +
     (32 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c32Allowed and c32Valid
    if c32Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c32Quality
        bestRank := c32Rank
        bestKind := c32Kind
        bestBias := c32Bias
        bestLen := 32
        bestUpperStart := c32UpperStart
        bestUpperEnd := c32UpperEnd
        bestLowerStart := c32LowerStart
        bestLowerEnd := c32LowerEnd
        bestUpperSlope := c32UpperSlope
        bestLowerSlope := c32LowerSlope
        bestPoleStart := c32PoleStart
        bestPoleEnd := c32PoleEnd
        bestPoleSize := c32PoleSize
        bestVolumeRatio := c32VolumeRatio
    else
        bool c32Distinct =
            na(bestLen) or
            math.abs(32 - bestLen) >= 8 or
            c32Kind != bestKind

        if c32Distinct and c32Rank > secondRank
            secondValid := true
            secondQuality := c32Quality
            secondRank := c32Rank
            secondKind := c32Kind
            secondBias := c32Bias
            secondLen := 32
            secondUpperStart := c32UpperStart
            secondUpperEnd := c32UpperEnd
            secondLowerStart := c32LowerStart
            secondLowerEnd := c32LowerEnd
            secondUpperSlope := c32UpperSlope
            secondLowerSlope := c32LowerSlope
            secondPoleStart := c32PoleStart
            secondPoleEnd := c32PoleEnd
            secondPoleSize := c32PoleSize
```

```
            secondVolumeRatio := c32VolumeRatio

    [c36Valid,
     c36Quality,
     c36Kind,
     c36Bias,
     c36UpperStart,
     c36UpperEnd,
     c36LowerStart,
     c36LowerEnd,
     c36UpperSlope,
     c36LowerSlope,
     c36PoleStart,
     c36PoleEnd,
     c36PoleSize,
     c36VolumeRatio] = f_evaluate(36)

    bool c36Allowed =
        36 >= effectiveMinLen and
        36 <= effectiveMaxLen

    float c36Rank =
        c36Quality +
        (c36Kind == 2 ? pennantPriorityBonus : 0.0) +
        (36 >= macroPatternLen ? macroPriorityBonus : 0.0)

    if c36Allowed and c36Valid
        if c36Rank > bestRank
            secondValid := bestValid
            secondQuality := bestQuality
            secondRank := bestRank
            secondKind := bestKind
            secondBias := bestBias
            secondLen := bestLen
            secondUpperStart := bestUpperStart
            secondUpperEnd := bestUpperEnd
            secondLowerStart := bestLowerStart
            secondLowerEnd := bestLowerEnd
            secondUpperSlope := bestUpperSlope
            secondLowerSlope := bestLowerSlope
            secondPoleStart := bestPoleStart
            secondPoleEnd := bestPoleEnd
            secondPoleSize := bestPoleSize
            secondVolumeRatio := bestVolumeRatio

            bestValid := true
            bestQuality := c36Quality
            bestRank := c36Rank
            bestKind := c36Kind
            bestBias := c36Bias
            bestLen := 36
            bestUpperStart := c36UpperStart
            bestUpperEnd := c36UpperEnd
            bestLowerStart := c36LowerStart
            bestLowerEnd := c36LowerEnd
            bestUpperSlope := c36UpperSlope
            bestLowerSlope := c36LowerSlope
            bestPoleStart := c36PoleStart
            bestPoleEnd := c36PoleEnd
            bestPoleSize := c36PoleSize
```

```
        bestVolumeRatio := c36VolumeRatio
    else
        bool c36Distinct =
            na(bestLen) or
            math.abs(36 - bestLen) >= 8 or
            c36Kind != bestKind

        if c36Distinct and c36Rank > secondRank
            secondValid := true
            secondQuality := c36Quality
            secondRank := c36Rank
            secondKind := c36Kind
            secondBias := c36Bias
            secondLen := 36
            secondUpperStart := c36UpperStart
            secondUpperEnd := c36UpperEnd
            secondLowerStart := c36LowerStart
            secondLowerEnd := c36LowerEnd
            secondUpperSlope := c36UpperSlope
            secondLowerSlope := c36LowerSlope
            secondPoleStart := c36PoleStart
            secondPoleEnd := c36PoleEnd
            secondPoleSize := c36PoleSize
            secondVolumeRatio := c36VolumeRatio

[c40Valid,
 c40Quality,
 c40Kind,
 c40Bias,
 c40UpperStart,
 c40UpperEnd,
 c40LowerStart,
 c40LowerEnd,
 c40UpperSlope,
 c40LowerSlope,
 c40PoleStart,
 c40PoleEnd,
 c40PoleSize,
 c40VolumeRatio] = f_evaluate(40)

bool c40Allowed =
    40 >= effectiveMinLen and
    40 <= effectiveMaxLen

float c40Rank =
    c40Quality +
    (c40Kind == 2 ? pennantPriorityBonus : 0.0) +
    (40 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c40Allowed and c40Valid
    if c40Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
```

```
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c40Quality
        bestRank := c40Rank
        bestKind := c40Kind
        bestBias := c40Bias
        bestLen := 40
        bestUpperStart := c40UpperStart
        bestUpperEnd := c40UpperEnd
        bestLowerStart := c40LowerStart
        bestLowerEnd := c40LowerEnd
        bestUpperSlope := c40UpperSlope
        bestLowerSlope := c40LowerSlope
        bestPoleStart := c40PoleStart
        bestPoleEnd := c40PoleEnd
        bestPoleSize := c40PoleSize
        bestVolumeRatio := c40VolumeRatio
    else
        bool c40Distinct =
            na(bestLen) or
            math.abs(40 - bestLen) >= 8 or
            c40Kind != bestKind

        if c40Distinct and c40Rank > secondRank
            secondValid := true
            secondQuality := c40Quality
            secondRank := c40Rank
            secondKind := c40Kind
            secondBias := c40Bias
            secondLen := 40
            secondUpperStart := c40UpperStart
            secondUpperEnd := c40UpperEnd
            secondLowerStart := c40LowerStart
            secondLowerEnd := c40LowerEnd
            secondUpperSlope := c40UpperSlope
            secondLowerSlope := c40LowerSlope
            secondPoleStart := c40PoleStart
            secondPoleEnd := c40PoleEnd
            secondPoleSize := c40PoleSize
            secondVolumeRatio := c40VolumeRatio

[c44Valid,
 c44Quality,
 c44Kind,
 c44Bias,
 c44UpperStart,
 c44UpperEnd,
 c44LowerStart,
 c44LowerEnd,
 c44UpperSlope,
 c44LowerSlope,
 c44PoleStart,
 c44PoleEnd,
```

```
 c44PoleSize,
 c44VolumeRatio] = f_evaluate(44)

bool c44Allowed =
    44 >= effectiveMinLen and
    44 <= effectiveMaxLen

float c44Rank =
    c44Quality +
    (c44Kind == 2 ? pennantPriorityBonus : 0.0) +
    (44 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c44Allowed and c44Valid
   if c44Rank > bestRank
      secondValid := bestValid
      secondQuality := bestQuality
      secondRank := bestRank
      secondKind := bestKind
      secondBias := bestBias
      secondLen := bestLen
      secondUpperStart := bestUpperStart
      secondUpperEnd := bestUpperEnd
      secondLowerStart := bestLowerStart
      secondLowerEnd := bestLowerEnd
      secondUpperSlope := bestUpperSlope
      secondLowerSlope := bestLowerSlope
      secondPoleStart := bestPoleStart
      secondPoleEnd := bestPoleEnd
      secondPoleSize := bestPoleSize
      secondVolumeRatio := bestVolumeRatio

      bestValid := true
      bestQuality := c44Quality
      bestRank := c44Rank
      bestKind := c44Kind
      bestBias := c44Bias
      bestLen := 44
      bestUpperStart := c44UpperStart
      bestUpperEnd := c44UpperEnd
      bestLowerStart := c44LowerStart
      bestLowerEnd := c44LowerEnd
      bestUpperSlope := c44UpperSlope
      bestLowerSlope := c44LowerSlope
      bestPoleStart := c44PoleStart
      bestPoleEnd := c44PoleEnd
      bestPoleSize := c44PoleSize
      bestVolumeRatio := c44VolumeRatio
   else
      bool c44Distinct =
          na(bestLen) or
          math.abs(44 - bestLen) >= 8 or
          c44Kind != bestKind

      if c44Distinct and c44Rank > secondRank
         secondValid := true
         secondQuality := c44Quality
         secondRank := c44Rank
         secondKind := c44Kind
         secondBias := c44Bias
         secondLen := 44
```

```
            secondUpperStart := c44UpperStart
            secondUpperEnd := c44UpperEnd
            secondLowerStart := c44LowerStart
            secondLowerEnd := c44LowerEnd
            secondUpperSlope := c44UpperSlope
            secondLowerSlope := c44LowerSlope
            secondPoleStart := c44PoleStart
            secondPoleEnd := c44PoleEnd
            secondPoleSize := c44PoleSize
            secondVolumeRatio := c44VolumeRatio

[c48Valid,
 c48Quality,
 c48Kind,
 c48Bias,
 c48UpperStart,
 c48UpperEnd,
 c48LowerStart,
 c48LowerEnd,
 c48UpperSlope,
 c48LowerSlope,
 c48PoleStart,
 c48PoleEnd,
 c48PoleSize,
 c48VolumeRatio] = f_evaluate(48)

bool c48Allowed =
     48 >= effectiveMinLen and
     48 <= effectiveMaxLen

float c48Rank =
     c48Quality +
     (c48Kind == 2 ? pennantPriorityBonus : 0.0) +
     (48 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c48Allowed and c48Valid
    if c48Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c48Quality
        bestRank := c48Rank
        bestKind := c48Kind
        bestBias := c48Bias
        bestLen := 48
```

```
        bestUpperStart := c48UpperStart
        bestUpperEnd := c48UpperEnd
        bestLowerStart := c48LowerStart
        bestLowerEnd := c48LowerEnd
        bestUpperSlope := c48UpperSlope
        bestLowerSlope := c48LowerSlope
        bestPoleStart := c48PoleStart
        bestPoleEnd := c48PoleEnd
        bestPoleSize := c48PoleSize
        bestVolumeRatio := c48VolumeRatio
    else
        bool c48Distinct =
            na(bestLen) or
            math.abs(48 - bestLen) >= 8 or
            c48Kind != bestKind

        if c48Distinct and c48Rank > secondRank
            secondValid := true
            secondQuality := c48Quality
            secondRank := c48Rank
            secondKind := c48Kind
            secondBias := c48Bias
            secondLen := 48
            secondUpperStart := c48UpperStart
            secondUpperEnd := c48UpperEnd
            secondLowerStart := c48LowerStart
            secondLowerEnd := c48LowerEnd
            secondUpperSlope := c48UpperSlope
            secondLowerSlope := c48LowerSlope
            secondPoleStart := c48PoleStart
            secondPoleEnd := c48PoleEnd
            secondPoleSize := c48PoleSize
            secondVolumeRatio := c48VolumeRatio

[c52Valid,
 c52Quality,
 c52Kind,
 c52Bias,
 c52UpperStart,
 c52UpperEnd,
 c52LowerStart,
 c52LowerEnd,
 c52UpperSlope,
 c52LowerSlope,
 c52PoleStart,
 c52PoleEnd,
 c52PoleSize,
 c52VolumeRatio] = f_evaluate(52)

bool c52Allowed =
    52 >= effectiveMinLen and
    52 <= effectiveMaxLen

float c52Rank =
    c52Quality +
    (c52Kind == 2 ? pennantPriorityBonus : 0.0) +
    (52 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c52Allowed and c52Valid
    if c52Rank > bestRank
```

```
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c52Quality
        bestRank := c52Rank
        bestKind := c52Kind
        bestBias := c52Bias
        bestLen := 52
        bestUpperStart := c52UpperStart
        bestUpperEnd := c52UpperEnd
        bestLowerStart := c52LowerStart
        bestLowerEnd := c52LowerEnd
        bestUpperSlope := c52UpperSlope
        bestLowerSlope := c52LowerSlope
        bestPoleStart := c52PoleStart
        bestPoleEnd := c52PoleEnd
        bestPoleSize := c52PoleSize
        bestVolumeRatio := c52VolumeRatio
    else
        bool c52Distinct =
            na(bestLen) or
            math.abs(52 - bestLen) >= 8 or
            c52Kind != bestKind

        if c52Distinct and c52Rank > secondRank
            secondValid := true
            secondQuality := c52Quality
            secondRank := c52Rank
            secondKind := c52Kind
            secondBias := c52Bias
            secondLen := 52
            secondUpperStart := c52UpperStart
            secondUpperEnd := c52UpperEnd
            secondLowerStart := c52LowerStart
            secondLowerEnd := c52LowerEnd
            secondUpperSlope := c52UpperSlope
            secondLowerSlope := c52LowerSlope
            secondPoleStart := c52PoleStart
            secondPoleEnd := c52PoleEnd
            secondPoleSize := c52PoleSize
            secondVolumeRatio := c52VolumeRatio

[c56Valid,
 c56Quality,
 c56Kind,
```

```
 c56Bias,
 c56UpperStart,
 c56UpperEnd,
 c56LowerStart,
 c56LowerEnd,
 c56UpperSlope,
 c56LowerSlope,
 c56PoleStart,
 c56PoleEnd,
 c56PoleSize,
 c56VolumeRatio] = f_evaluate(56)

bool c56Allowed =
     56 >= effectiveMinLen and
     56 <= effectiveMaxLen

float c56Rank =
     c56Quality +
     (c56Kind == 2 ? pennantPriorityBonus : 0.0) +
     (56 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c56Allowed and c56Valid
    if c56Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c56Quality
        bestRank := c56Rank
        bestKind := c56Kind
        bestBias := c56Bias
        bestLen := 56
        bestUpperStart := c56UpperStart
        bestUpperEnd := c56UpperEnd
        bestLowerStart := c56LowerStart
        bestLowerEnd := c56LowerEnd
        bestUpperSlope := c56UpperSlope
        bestLowerSlope := c56LowerSlope
        bestPoleStart := c56PoleStart
        bestPoleEnd := c56PoleEnd
        bestPoleSize := c56PoleSize
        bestVolumeRatio := c56VolumeRatio
    else
        bool c56Distinct =
            na(bestLen) or
            math.abs(56 - bestLen) >= 8 or
```

```
            c56Kind != bestKind

        if c56Distinct and c56Rank > secondRank
            secondValid := true
            secondQuality := c56Quality
            secondRank := c56Rank
            secondKind := c56Kind
            secondBias := c56Bias
            secondLen := 56
            secondUpperStart := c56UpperStart
            secondUpperEnd := c56UpperEnd
            secondLowerStart := c56LowerStart
            secondLowerEnd := c56LowerEnd
            secondUpperSlope := c56UpperSlope
            secondLowerSlope := c56LowerSlope
            secondPoleStart := c56PoleStart
            secondPoleEnd := c56PoleEnd
            secondPoleSize := c56PoleSize
            secondVolumeRatio := c56VolumeRatio

[c60Valid,
 c60Quality,
 c60Kind,
 c60Bias,
 c60UpperStart,
 c60UpperEnd,
 c60LowerStart,
 c60LowerEnd,
 c60UpperSlope,
 c60LowerSlope,
 c60PoleStart,
 c60PoleEnd,
 c60PoleSize,
 c60VolumeRatio] = f_evaluate(60)

bool c60Allowed =
    60 >= effectiveMinLen and
    60 <= effectiveMaxLen

float c60Rank =
    c60Quality +
    (c60Kind == 2 ? pennantPriorityBonus : 0.0) +
    (60 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c60Allowed and c60Valid
   if c60Rank > bestRank
       secondValid := bestValid
       secondQuality := bestQuality
       secondRank := bestRank
       secondKind := bestKind
       secondBias := bestBias
       secondLen := bestLen
       secondUpperStart := bestUpperStart
       secondUpperEnd := bestUpperEnd
       secondLowerStart := bestLowerStart
       secondLowerEnd := bestLowerEnd
       secondUpperSlope := bestUpperSlope
       secondLowerSlope := bestLowerSlope
       secondPoleStart := bestPoleStart
       secondPoleEnd := bestPoleEnd
```

```
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c60Quality
        bestRank := c60Rank
        bestKind := c60Kind
        bestBias := c60Bias
        bestLen := 60
        bestUpperStart := c60UpperStart
        bestUpperEnd := c60UpperEnd
        bestLowerStart := c60LowerStart
        bestLowerEnd := c60LowerEnd
        bestUpperSlope := c60UpperSlope
        bestLowerSlope := c60LowerSlope
        bestPoleStart := c60PoleStart
        bestPoleEnd := c60PoleEnd
        bestPoleSize := c60PoleSize
        bestVolumeRatio := c60VolumeRatio
    else
        bool c60Distinct =
            na(bestLen) or
            math.abs(60 - bestLen) >= 8 or
            c60Kind != bestKind

        if c60Distinct and c60Rank > secondRank
            secondValid := true
            secondQuality := c60Quality
            secondRank := c60Rank
            secondKind := c60Kind
            secondBias := c60Bias
            secondLen := 60
            secondUpperStart := c60UpperStart
            secondUpperEnd := c60UpperEnd
            secondLowerStart := c60LowerStart
            secondLowerEnd := c60LowerEnd
            secondUpperSlope := c60UpperSlope
            secondLowerSlope := c60LowerSlope
            secondPoleStart := c60PoleStart
            secondPoleEnd := c60PoleEnd
            secondPoleSize := c60PoleSize
            secondVolumeRatio := c60VolumeRatio

[c64Valid,
 c64Quality,
 c64Kind,
 c64Bias,
 c64UpperStart,
 c64UpperEnd,
 c64LowerStart,
 c64LowerEnd,
 c64UpperSlope,
 c64LowerSlope,
 c64PoleStart,
 c64PoleEnd,
 c64PoleSize,
 c64VolumeRatio] = f_evaluate(64)

bool c64Allowed =
    64 >= effectiveMinLen and
```

```
    64 <= effectiveMaxLen

float c64Rank =
    c64Quality +
    (c64Kind == 2 ? pennantPriorityBonus : 0.0) +
    (64 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c64Allowed and c64Valid
   if c64Rank > bestRank
      secondValid := bestValid
      secondQuality := bestQuality
      secondRank := bestRank
      secondKind := bestKind
      secondBias := bestBias
      secondLen := bestLen
      secondUpperStart := bestUpperStart
      secondUpperEnd := bestUpperEnd
      secondLowerStart := bestLowerStart
      secondLowerEnd := bestLowerEnd
      secondUpperSlope := bestUpperSlope
      secondLowerSlope := bestLowerSlope
      secondPoleStart := bestPoleStart
      secondPoleEnd := bestPoleEnd
      secondPoleSize := bestPoleSize
      secondVolumeRatio := bestVolumeRatio

      bestValid := true
      bestQuality := c64Quality
      bestRank := c64Rank
      bestKind := c64Kind
      bestBias := c64Bias
      bestLen := 64
      bestUpperStart := c64UpperStart
      bestUpperEnd := c64UpperEnd
      bestLowerStart := c64LowerStart
      bestLowerEnd := c64LowerEnd
      bestUpperSlope := c64UpperSlope
      bestLowerSlope := c64LowerSlope
      bestPoleStart := c64PoleStart
      bestPoleEnd := c64PoleEnd
      bestPoleSize := c64PoleSize
      bestVolumeRatio := c64VolumeRatio
   else
      bool c64Distinct =
          na(bestLen) or
          math.abs(64 - bestLen) >= 8 or
          c64Kind != bestKind

      if c64Distinct and c64Rank > secondRank
         secondValid := true
         secondQuality := c64Quality
         secondRank := c64Rank
         secondKind := c64Kind
         secondBias := c64Bias
         secondLen := 64
         secondUpperStart := c64UpperStart
         secondUpperEnd := c64UpperEnd
         secondLowerStart := c64LowerStart
         secondLowerEnd := c64LowerEnd
         secondUpperSlope := c64UpperSlope
```

```
            secondLowerSlope := c64LowerSlope
            secondPoleStart := c64PoleStart
            secondPoleEnd := c64PoleEnd
            secondPoleSize := c64PoleSize
            secondVolumeRatio := c64VolumeRatio

[c68Valid,
 c68Quality,
 c68Kind,
 c68Bias,
 c68UpperStart,
 c68UpperEnd,
 c68LowerStart,
 c68LowerEnd,
 c68UpperSlope,
 c68LowerSlope,
 c68PoleStart,
 c68PoleEnd,
 c68PoleSize,
 c68VolumeRatio] = f_evaluate(68)

bool c68Allowed =
    68 >= effectiveMinLen and
    68 <= effectiveMaxLen

float c68Rank =
    c68Quality +
    (c68Kind == 2 ? pennantPriorityBonus : 0.0) +
    (68 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c68Allowed and c68Valid
   if c68Rank > bestRank
       secondValid := bestValid
       secondQuality := bestQuality
       secondRank := bestRank
       secondKind := bestKind
       secondBias := bestBias
       secondLen := bestLen
       secondUpperStart := bestUpperStart
       secondUpperEnd := bestUpperEnd
       secondLowerStart := bestLowerStart
       secondLowerEnd := bestLowerEnd
       secondUpperSlope := bestUpperSlope
       secondLowerSlope := bestLowerSlope
       secondPoleStart := bestPoleStart
       secondPoleEnd := bestPoleEnd
       secondPoleSize := bestPoleSize
       secondVolumeRatio := bestVolumeRatio

       bestValid := true
       bestQuality := c68Quality
       bestRank := c68Rank
       bestKind := c68Kind
       bestBias := c68Bias
       bestLen := 68
       bestUpperStart := c68UpperStart
       bestUpperEnd := c68UpperEnd
       bestLowerStart := c68LowerStart
       bestLowerEnd := c68LowerEnd
       bestUpperSlope := c68UpperSlope
```

```
        bestLowerSlope := c68LowerSlope
        bestPoleStart := c68PoleStart
        bestPoleEnd := c68PoleEnd
        bestPoleSize := c68PoleSize
        bestVolumeRatio := c68VolumeRatio
    else
        bool c68Distinct =
            na(bestLen) or
            math.abs(68 - bestLen) >= 8 or
            c68Kind != bestKind

        if c68Distinct and c68Rank > secondRank
            secondValid := true
            secondQuality := c68Quality
            secondRank := c68Rank
            secondKind := c68Kind
            secondBias := c68Bias
            secondLen := 68
            secondUpperStart := c68UpperStart
            secondUpperEnd := c68UpperEnd
            secondLowerStart := c68LowerStart
            secondLowerEnd := c68LowerEnd
            secondUpperSlope := c68UpperSlope
            secondLowerSlope := c68LowerSlope
            secondPoleStart := c68PoleStart
            secondPoleEnd := c68PoleEnd
            secondPoleSize := c68PoleSize
            secondVolumeRatio := c68VolumeRatio

[c72Valid,
 c72Quality,
 c72Kind,
 c72Bias,
 c72UpperStart,
 c72UpperEnd,
 c72LowerStart,
 c72LowerEnd,
 c72UpperSlope,
 c72LowerSlope,
 c72PoleStart,
 c72PoleEnd,
 c72PoleSize,
 c72VolumeRatio] = f_evaluate(72)

bool c72Allowed =
    72 >= effectiveMinLen and
    72 <= effectiveMaxLen

float c72Rank =
    c72Quality +
    (c72Kind == 2 ? pennantPriorityBonus : 0.0) +
    (72 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c72Allowed and c72Valid
    if c72Rank > bestRank
        secondValid := bestValid
        secondQuality := bestQuality
        secondRank := bestRank
        secondKind := bestKind
        secondBias := bestBias
```

```
        secondLen := bestLen
        secondUpperStart := bestUpperStart
        secondUpperEnd := bestUpperEnd
        secondLowerStart := bestLowerStart
        secondLowerEnd := bestLowerEnd
        secondUpperSlope := bestUpperSlope
        secondLowerSlope := bestLowerSlope
        secondPoleStart := bestPoleStart
        secondPoleEnd := bestPoleEnd
        secondPoleSize := bestPoleSize
        secondVolumeRatio := bestVolumeRatio

        bestValid := true
        bestQuality := c72Quality
        bestRank := c72Rank
        bestKind := c72Kind
        bestBias := c72Bias
        bestLen := 72
        bestUpperStart := c72UpperStart
        bestUpperEnd := c72UpperEnd
        bestLowerStart := c72LowerStart
        bestLowerEnd := c72LowerEnd
        bestUpperSlope := c72UpperSlope
        bestLowerSlope := c72LowerSlope
        bestPoleStart := c72PoleStart
        bestPoleEnd := c72PoleEnd
        bestPoleSize := c72PoleSize
        bestVolumeRatio := c72VolumeRatio
    else
        bool c72Distinct =
            na(bestLen) or
            math.abs(72 - bestLen) >= 8 or
            c72Kind != bestKind

        if c72Distinct and c72Rank > secondRank
            secondValid := true
            secondQuality := c72Quality
            secondRank := c72Rank
            secondKind := c72Kind
            secondBias := c72Bias
            secondLen := 72
            secondUpperStart := c72UpperStart
            secondUpperEnd := c72UpperEnd
            secondLowerStart := c72LowerStart
            secondLowerEnd := c72LowerEnd
            secondUpperSlope := c72UpperSlope
            secondLowerSlope := c72LowerSlope
            secondPoleStart := c72PoleStart
            secondPoleEnd := c72PoleEnd
            secondPoleSize := c72PoleSize
            secondVolumeRatio := c72VolumeRatio

[c76Valid,
 c76Quality,
 c76Kind,
 c76Bias,
 c76UpperStart,
 c76UpperEnd,
 c76LowerStart,
 c76LowerEnd,
```

```
 c76UpperSlope,
 c76LowerSlope,
 c76PoleStart,
 c76PoleEnd,
 c76PoleSize,
 c76VolumeRatio] = f_evaluate(76)

bool c76Allowed =
    76 >= effectiveMinLen and
    76 <= effectiveMaxLen

float c76Rank =
    c76Quality +
    (c76Kind == 2 ? pennantPriorityBonus : 0.0) +
    (76 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c76Allowed and c76Valid
   if c76Rank > bestRank
      secondValid := bestValid
      secondQuality := bestQuality
      secondRank := bestRank
      secondKind := bestKind
      secondBias := bestBias
      secondLen := bestLen
      secondUpperStart := bestUpperStart
      secondUpperEnd := bestUpperEnd
      secondLowerStart := bestLowerStart
      secondLowerEnd := bestLowerEnd
      secondUpperSlope := bestUpperSlope
      secondLowerSlope := bestLowerSlope
      secondPoleStart := bestPoleStart
      secondPoleEnd := bestPoleEnd
      secondPoleSize := bestPoleSize
      secondVolumeRatio := bestVolumeRatio

      bestValid := true
      bestQuality := c76Quality
      bestRank := c76Rank
      bestKind := c76Kind
      bestBias := c76Bias
      bestLen := 76
      bestUpperStart := c76UpperStart
      bestUpperEnd := c76UpperEnd
      bestLowerStart := c76LowerStart
      bestLowerEnd := c76LowerEnd
      bestUpperSlope := c76UpperSlope
      bestLowerSlope := c76LowerSlope
      bestPoleStart := c76PoleStart
      bestPoleEnd := c76PoleEnd
      bestPoleSize := c76PoleSize
      bestVolumeRatio := c76VolumeRatio
   else
      bool c76Distinct =
          na(bestLen) or
          math.abs(76 - bestLen) >= 8 or
          c76Kind != bestKind

      if c76Distinct and c76Rank > secondRank
         secondValid := true
         secondQuality := c76Quality
```

```
            secondRank := c76Rank
            secondKind := c76Kind
            secondBias := c76Bias
            secondLen := 76
            secondUpperStart := c76UpperStart
            secondUpperEnd := c76UpperEnd
            secondLowerStart := c76LowerStart
            secondLowerEnd := c76LowerEnd
            secondUpperSlope := c76UpperSlope
            secondLowerSlope := c76LowerSlope
            secondPoleStart := c76PoleStart
            secondPoleEnd := c76PoleEnd
            secondPoleSize := c76PoleSize
            secondVolumeRatio := c76VolumeRatio

[c80Valid,
 c80Quality,
 c80Kind,
 c80Bias,
 c80UpperStart,
 c80UpperEnd,
 c80LowerStart,
 c80LowerEnd,
 c80UpperSlope,
 c80LowerSlope,
 c80PoleStart,
 c80PoleEnd,
 c80PoleSize,
 c80VolumeRatio] = f_evaluate(80)

bool c80Allowed =
    80 >= effectiveMinLen and
    80 <= effectiveMaxLen

float c80Rank =
    c80Quality +
    (c80Kind == 2 ? pennantPriorityBonus : 0.0) +
    (80 >= macroPatternLen ? macroPriorityBonus : 0.0)

if c80Allowed and c80Valid
   if c80Rank > bestRank
       secondValid := bestValid
       secondQuality := bestQuality
       secondRank := bestRank
       secondKind := bestKind
       secondBias := bestBias
       secondLen := bestLen
       secondUpperStart := bestUpperStart
       secondUpperEnd := bestUpperEnd
       secondLowerStart := bestLowerStart
       secondLowerEnd := bestLowerEnd
       secondUpperSlope := bestUpperSlope
       secondLowerSlope := bestLowerSlope
       secondPoleStart := bestPoleStart
       secondPoleEnd := bestPoleEnd
       secondPoleSize := bestPoleSize
       secondVolumeRatio := bestVolumeRatio

       bestValid := true
       bestQuality := c80Quality
```

```
            bestRank := c80Rank
            bestKind := c80Kind
            bestBias := c80Bias
            bestLen := 80
            bestUpperStart := c80UpperStart
            bestUpperEnd := c80UpperEnd
            bestLowerStart := c80LowerStart
            bestLowerEnd := c80LowerEnd
            bestUpperSlope := c80UpperSlope
            bestLowerSlope := c80LowerSlope
            bestPoleStart := c80PoleStart
            bestPoleEnd := c80PoleEnd
            bestPoleSize := c80PoleSize
            bestVolumeRatio := c80VolumeRatio
        else
            bool c80Distinct =
                na(bestLen) or
                math.abs(80 - bestLen) >= 8 or
                c80Kind != bestKind

            if c80Distinct and c80Rank > secondRank
                secondValid := true
                secondQuality := c80Quality
                secondRank := c80Rank
                secondKind := c80Kind
                secondBias := c80Bias
                secondLen := 80
                secondUpperStart := c80UpperStart
                secondUpperEnd := c80UpperEnd
                secondLowerStart := c80LowerStart
                secondLowerEnd := c80LowerEnd
                secondUpperSlope := c80UpperSlope
                secondLowerSlope := c80LowerSlope
                secondPoleStart := c80PoleStart
                secondPoleEnd := c80PoleEnd
                secondPoleSize := c80PoleSize
                secondVolumeRatio := c80VolumeRatio

bool developingPattern =
    bestValid and
    bestQuality >= minDevelopingQuality

float upperNow =
    developingPattern ?
    bestUpperEnd + bestUpperSlope :
    na

float lowerNow =
    developingPattern ?
    bestLowerEnd + bestLowerSlope :
    na

float closingSpeed =
    developingPattern and bestKind == 2 ?
    bestLowerSlope - bestUpperSlope :
    0.0

float widthNow =
    developingPattern ?
    upperNow - lowerNow :
```

```
    na

int apexAhead =
    developingPattern and
    bestKind == 2 and
    closingSpeed > syminfo.mintick ?
    int(
        math.max(
            2.0,
            math.min(
                maxApexProjection,
                math.round(widthNow / closingSpeed)))) :
    projectionBars

float upperProjected =
    developingPattern ?
    upperNow + bestUpperSlope * apexAhead :
    na

float lowerProjected =
    developingPattern ?
    lowerNow + bestLowerSlope * apexAhead :
    na


bool secondDevelopingPattern =
    showSecondPattern and
    secondValid and
    secondQuality >= minDevelopingQuality + 8.0 and
    (
        na(bestLen) or
        math.abs(secondLen - bestLen) >= 12 or
        secondKind != bestKind
    )

float secondUpperNow =
    secondDevelopingPattern ?
    secondUpperEnd + secondUpperSlope :
    na

float secondLowerNow =
    secondDevelopingPattern ?
    secondLowerEnd + secondLowerSlope :
    na

float secondClosingSpeed =
    secondDevelopingPattern and secondKind == 2 ?
    secondLowerSlope - secondUpperSlope :
    0.0

float secondWidthNow =
    secondDevelopingPattern ?
    secondUpperNow - secondLowerNow :
    na

int secondApexAhead =
    secondDevelopingPattern and
    secondKind == 2 and
    secondClosingSpeed > syminfo.mintick ?
    int(
```

```
        math.max(
            2.0,
            math.min(
                maxApexProjection,
                math.round(secondWidthNow / secondClosingSpeed)))) :
    projectionBars

float secondUpperProjected =
    secondDevelopingPattern ?
    secondUpperNow + secondUpperSlope * secondApexAhead :
    na

float secondLowerProjected =
    secondDevelopingPattern ?
    secondLowerNow + secondLowerSlope * secondApexAhead :
    na

//
------------------------------------------------------------------------------------------------------------
// Активная формирующаяся фигура
//
------------------------------------------------------------------------------------------------------------

var line activePole = na
var line activeUpper = na
var line activeLower = na
var label activeLabel = na

var line secondActivePole = na
var line secondActiveUpper = na
var line secondActiveLower = na
var label secondActiveLabel = na

f_deleteActive() =>
    if not na(activePole)
        line.delete(activePole)

    if not na(activeUpper)
        line.delete(activeUpper)

    if not na(activeLower)
        line.delete(activeLower)

    if not na(activeLabel)
        label.delete(activeLabel)

    true

f_deleteSecondActive() =>
    if not na(secondActivePole)
        line.delete(secondActivePole)

    if not na(secondActiveUpper)
        line.delete(secondActiveUpper)

    if not na(secondActiveLower)
        line.delete(secondActiveLower)
```

```
    if not na(secondActiveLabel)
        label.delete(secondActiveLabel)

    true

if barstate.islast
    if showDeveloping and developingPattern
        int patternStartX = bar_index - bestLen
        int poleStartX = patternStartX - poleLen
        int projectedX = bar_index + apexAhead

        color activePatternColor =
             bestKind == 2 ?
             (bestBias == 1 ? pennantLongColor : pennantShortColor) :
             (bestBias == 1 ? longColor : shortColor)

        color activeBoundaryColor =
             color.new(activePatternColor, formingPatternTransparency)

        if na(activePole)
            activePole := line.new(
                 poleStartX,
                 bestPoleStart,
                 patternStartX,
                 bestPoleEnd,
                 xloc=xloc.bar_index,
                 color=activePatternColor,
                 width=3)
        else
            line.set_xy1(
                 activePole,
                 poleStartX,
                 bestPoleStart)
            line.set_xy2(
                 activePole,
                 patternStartX,
                 bestPoleEnd)
            line.set_color(
                 activePole,
                 activePatternColor)

        if na(activeUpper)
            activeUpper := line.new(
                 patternStartX,
                 bestUpperStart,
                 projectedX,
                 upperProjected,
                 xloc=xloc.bar_index,
                 color=activeBoundaryColor,
                 style=line.style_dashed,
                 width=2)
        else
            line.set_xy1(
                 activeUpper,
                 patternStartX,
                 bestUpperStart)
            line.set_xy2(
                 activeUpper,
                 projectedX,
                 upperProjected)
```

```
    line.set_color(
        activeUpper,
        activeBoundaryColor)
    line.set_style(
        activeUpper,
        line.style_dashed)

if na(activeLower)
    activeLower := line.new(
        patternStartX,
        bestLowerStart,
        projectedX,
        lowerProjected,
        xloc=xloc.bar_index,
        color=activeBoundaryColor,
        style=line.style_dashed,
        width=2)
else
    line.set_xy1(
        activeLower,
        patternStartX,
        bestLowerStart)
    line.set_xy2(
        activeLower,
        projectedX,
        lowerProjected)
    line.set_color(
        activeLower,
        activeBoundaryColor)
    line.set_style(
        activeLower,
        line.style_dashed)

if not na(activeLabel)
    label.delete(activeLabel)

string directionText =
     bestBias == 1 ?
     "ЛОНГ" :
     "ШОРТ"

string patternText =
     bestKind == 2 ?
     "ВЫМПЕЛ" :
     "ФЛАГ"

string activeText =
     "EVA · " +
     directionText +
     " " +
     patternText +
     "\nФОРМИРУЕТСЯ · КАЧ. " +
     str.tostring(bestQuality, "#") +
     "% · " +
     str.tostring(bestLen) +
     " баров"

activeLabel := label.new(
     x=bar_index,
     y=bestBias == 1 ? lowerNow : upperNow,
```

```
        text=activeText,
        xloc=xloc.bar_index,
        yloc=bestBias == 1 ? yloc.belowbar : yloc.abovebar,
        style=bestBias == 1 ?
            label.style_label_up :
            label.style_label_down,
        color=color.new(activePatternColor, 8),
        textcolor=color.rgb(15, 23, 42),
        size=size.tiny)
    if secondDevelopingPattern
        int secondPatternStartX = bar_index - secondLen
        int secondPoleStartX = secondPatternStartX - poleLen
        int secondProjectedX = bar_index + secondApexAhead

        color secondBaseColor =
            secondKind == 2 ?
            (secondBias == 1 ? pennantLongColor : pennantShortColor) :
            (secondBias == 1 ? longColor : shortColor)

        color secondDirectionColor =
            color.new(secondBaseColor, 28)

        color secondBoundaryColor =
            color.new(
                secondBaseColor,
                math.min(formingPatternTransparency + 15, 90))

        if na(secondActivePole)
            secondActivePole := line.new(
                secondPoleStartX,
                secondPoleStart,
                secondPatternStartX,
                secondPoleEnd,
                xloc=xloc.bar_index,
                color=secondDirectionColor,
                width=2)
        else
            line.set_xy1(secondActivePole, secondPoleStartX, secondPoleStart)
            line.set_xy2(secondActivePole, secondPatternStartX, secondPoleEnd)
            line.set_color(secondActivePole, secondDirectionColor)

        if na(secondActiveUpper)
            secondActiveUpper := line.new(
                secondPatternStartX,
                secondUpperStart,
                secondProjectedX,
                secondUpperProjected,
                xloc=xloc.bar_index,
                color=secondBoundaryColor,
                style=line.style_dotted,
                width=2)
        else
            line.set_xy1(secondActiveUpper, secondPatternStartX, secondUpperStart)
            line.set_xy2(secondActiveUpper, secondProjectedX, secondUpperProjected)
            line.set_color(secondActiveUpper, secondBoundaryColor)
            line.set_style(secondActiveUpper, line.style_dotted)

        if na(secondActiveLower)
            secondActiveLower := line.new(
                secondPatternStartX,
```

```
            secondLowerStart,
            secondProjectedX,
            secondLowerProjected,
            xloc=xloc.bar_index,
            color=secondBoundaryColor,
            style=line.style_dotted,
            width=2)
        else
            line.set_xy1(secondActiveLower, secondPatternStartX, secondLowerStart)
            line.set_xy2(secondActiveLower, secondProjectedX, secondLowerProjected)
            line.set_color(secondActiveLower, secondBoundaryColor)
            line.set_style(secondActiveLower, line.style_dotted)

        if not na(secondActiveLabel)
            label.delete(secondActiveLabel)

        string secondDirectionText =
             secondBias == 1 ?
             "ЛОНГ" :
             "ШОРТ"

        string secondPatternText =
             secondKind == 2 ?
             "ВЫМПЕЛ" :
             "ФЛАГ"

        string secondText =
             "EVA · " +
             secondDirectionText +
             " " +
             secondPatternText +
             "\nДОП. ФИГУРА · КАЧ. " +
             str.tostring(secondQuality, "#") +
             "% · " +
             str.tostring(secondLen) +
             " баров"

        secondActiveLabel := label.new(
             x=bar_index,
             y=secondBias == 1 ? secondLowerNow : secondUpperNow,
             text=secondText,
             xloc=xloc.bar_index,
             yloc=secondBias == 1 ? yloc.belowbar : yloc.abovebar,
             style=secondBias == 1 ? label.style_label_up : label.style_label_down,
             color=color.new(secondBaseColor, 45),
             textcolor=color.rgb(15, 23, 42),
             size=size.tiny)
    else
        f_deleteSecondActive()
        secondActivePole := na
        secondActiveUpper := na
        secondActiveLower := na
        secondActiveLabel := na

else
    f_deleteActive()
    f_deleteSecondActive()

    activePole := na
    activeUpper := na
```

```
        activeLower := na
        activeLabel := na

        secondActivePole := na
        secondActiveUpper := na
        secondActiveLower := na
        secondActiveLabel := na

//
--------------------------------------------------------------------------------
--------------------------------------------------------------------
// Подтверждение пробоя
//
--------------------------------------------------------------------------------
--------------------------------------------------------------------

float averageBreakoutVolume =
    ta.sma(volume[1], breakoutVolumeLen)

float breakoutVolumeRatio =
    not na(averageBreakoutVolume) and
    averageBreakoutVolume > 0.0 ?
    volume / averageBreakoutVolume :
    1.0

bool breakoutVolumeOk =
    not useBreakoutVolume or
    breakoutVolumeRatio >= minBreakoutVolumeRatio

float longBreakLevel =
    developingPattern ?
    upperNow + breakoutBufferAtr * atr :
    na

float shortBreakLevel =
    developingPattern ?
    lowerNow - breakoutBufferAtr * atr :
    na

bool longBreakout =
    developingPattern and
    bestBias == 1 and
    close > longBreakLevel and
    close[1] <= bestUpperEnd +
        breakoutBufferAtr * atr[1]

bool shortBreakout =
    developingPattern and
    bestBias == -1 and
    close < shortBreakLevel and
    close[1] >= bestLowerEnd -
        breakoutBufferAtr * atr[1]

var int lastSignalBar = na

bool cooldownOk =
    na(lastSignalBar) or
    bar_index - lastSignalBar > signalCooldown

bool confirmedPattern =
```

```
    barstate.isconfirmed and
    developingPattern and
    bestQuality >= minConfirmedQuality and
    breakoutVolumeOk and
    cooldownOk and
    (longBreakout or shortBreakout)

bool confirmedLong =
    confirmedPattern and
    longBreakout

bool confirmedShort =
    confirmedPattern and
    shortBreakout

bool confirmedPennant =
    confirmedPattern and
    bestKind == 2

bool confirmedFlag =
    confirmedPattern and
    bestKind == 1

if confirmedPattern
    int patternStartX = bar_index - bestLen
    int poleStartX = patternStartX - poleLen
    int projectedX = bar_index + apexAhead
    color signalColor =
        confirmedPennant ?
        (confirmedLong ? pennantLongColor : pennantShortColor) :
        (confirmedLong ? longColor : shortColor)

    line poleLine = line.new(
        poleStartX,
        bestPoleStart,
        patternStartX,
        bestPoleEnd,
        xloc=xloc.bar_index,
        color=signalColor,
        width=3)

    line upperLine = line.new(
        patternStartX,
        bestUpperStart,
        projectedX,
        upperProjected,
        xloc=xloc.bar_index,
        color=signalColor,
        width=2)

    line lowerLine = line.new(
        patternStartX,
        bestLowerStart,
        projectedX,
        lowerProjected,
        xloc=xloc.bar_index,
        color=signalColor,
        width=2)

    f_keepLine(poleLine)
```

```
f_keepLine(upperLine)
f_keepLine(lowerLine)

if showTarget
    float targetPrice =
        confirmedLong ?
        close + bestPoleSize * targetFactor :
        close - bestPoleSize * targetFactor

    line targetLine = line.new(
        bar_index,
        targetPrice,
        bar_index + targetBars,
        targetPrice,
        xloc=xloc.bar_index,
        color=targetColor,
        style=line.style_dashed,
        width=1)

    f_keepLine(targetLine)

string directionText =
    confirmedLong ?
    "ЛОНГ" :
    "ШОРТ"

string patternText =
    confirmedPennant ?
    "ВЫМПЕЛ" :
    "ФЛАГ"

string signalText =
    "EVA · " +
    directionText +
    "\n" +
    patternText +
    " · КАЧ. " +
    str.tostring(bestQuality, "#") +
    "%"

label signalLabel = label.new(
    x=bar_index,
    y=confirmedLong ? low : high,
    text=signalText,
    xloc=xloc.bar_index,
    yloc=confirmedLong ?
        yloc.belowbar :
        yloc.abovebar,
    style=confirmedLong ?
        label.style_label_up :
        label.style_label_down,
    color=signalColor,
    textcolor=color.white,
    size=size.tiny)

f_keepLabel(signalLabel)
f_trimConfirmed()

lastSignalBar := bar_index
```

```
//
// ---------------------------------------------------------------------------------------------------------------------
// Разворотные фигуры: pivot engine
//
// ---------------------------------------------------------------------------------------------------------------------

type EvaPattern3
    bool active
    int x1
    float y1
    int x2
    float y2
    int x3
    float y3
    line leg1
    line leg2
    line neck

type EvaPattern5
    bool active
    int x1
    float y1
    int x2
    float y2
    int x3
    float y3
    int x4
    float y4
    int x5
    float y5
    line leg1
    line leg2
    line leg3
    line leg4
    line neck

var array<int> reversalTypes = array.new_int()
var array<int> reversalBars = array.new_int()
var array<float> reversalPrices = array.new_float()

var array<int> macroReversalTypes = array.new_int()
var array<int> macroReversalBars = array.new_int()
var array<float> macroReversalPrices = array.new_float()

var EvaPattern3 pendingDoubleBottom =
     EvaPattern3.new(false, na, na, na, na, na, na, na, na, na)

var EvaPattern3 pendingDoubleTop =
     EvaPattern3.new(false, na, na, na, na, na, na, na, na, na)

var EvaPattern5 pendingHeadShoulders =
     EvaPattern5.new(false, na, na, na, na, na, na, na, na, na, na, na, na, na, na, na, na)

var EvaPattern5 pendingInverseHeadShoulders =
     EvaPattern5.new(false, na, na, na, na, na, na, na, na, na, na, na, na, na, na, na, na)

var int lastDoubleBottomPivot = na
```

```
var int lastDoubleTopPivot = na
var int lastHeadShouldersPivot = na
var int lastInverseHeadShouldersPivot = na

var bool pendingDoubleBottomIsMacro = false
var bool pendingDoubleTopIsMacro = false
var bool pendingHeadShouldersIsMacro = false
var bool pendingInverseHeadShouldersIsMacro = false

var float pendingDoubleBottomQuality = na
var float pendingDoubleTopQuality = na
var float pendingHeadShouldersQuality = na
var float pendingInverseHeadShouldersQuality = na

f_addReversalPivot(
     int pivotType,
     int pivotBar,
     float pivotPrice) =>
    bool changed = false
    int count = array.size(reversalTypes)

    if count == 0
        array.push(reversalTypes, pivotType)
        array.push(reversalBars, pivotBar)
        array.push(reversalPrices, pivotPrice)
        changed := true
    else
        int lastIndex = count - 1
        int lastType = array.get(reversalTypes, lastIndex)
        float lastPrice = array.get(reversalPrices, lastIndex)

        if lastType == pivotType
            bool replaceLast =
                 (pivotType == 1 and pivotPrice > lastPrice) or
                 (pivotType == -1 and pivotPrice < lastPrice)

            if replaceLast
                array.set(reversalBars, lastIndex, pivotBar)
                array.set(reversalPrices, lastIndex, pivotPrice)
                changed := true
        else
            array.push(reversalTypes, pivotType)
            array.push(reversalBars, pivotBar)
            array.push(reversalPrices, pivotPrice)
            changed := true

    while array.size(reversalTypes) > 20
        int removedType = array.shift(reversalTypes)
        int removedBar = array.shift(reversalBars)
        float removedPrice = array.shift(reversalPrices)

    changed

f_addMacroReversalPivot(
     int pivotType,
     int pivotBar,
     float pivotPrice) =>
    bool changed = false
    int count = array.size(macroReversalTypes)
```

```
    if count == 0
        array.push(macroReversalTypes, pivotType)
        array.push(macroReversalBars, pivotBar)
        array.push(macroReversalPrices, pivotPrice)
        changed := true
    else
        int lastIndex = count - 1
        int lastType = array.get(macroReversalTypes, lastIndex)
        float lastPrice = array.get(macroReversalPrices, lastIndex)

        if lastType == pivotType
            bool replaceLast =
                (pivotType == 1 and pivotPrice > lastPrice) or
                (pivotType == -1 and pivotPrice < lastPrice)

            if replaceLast
                array.set(macroReversalBars, lastIndex, pivotBar)
                array.set(macroReversalPrices, lastIndex, pivotPrice)
                changed := true
        else
            array.push(macroReversalTypes, pivotType)
            array.push(macroReversalBars, pivotBar)
            array.push(macroReversalPrices, pivotPrice)
            changed := true

    while array.size(macroReversalTypes) > 20
        int removedType = array.shift(macroReversalTypes)
        int removedBar = array.shift(macroReversalBars)
        float removedPrice = array.shift(macroReversalPrices)

    changed

f_necklineValue(
     int x1,
     float y1,
     int x2,
     float y2,
     int x) =>
    float dx = math.max(x2 - x1, 1)
    float slope = (y2 - y1) / dx
    y1 + slope * (x - x1)


f_clamp01(float value) =>
    math.max(0.0, math.min(1.0, value))

f_timeQuality(
     float ratio,
     float maxRatio) =>
    maxRatio <= 1.0 ?
     (ratio <= 1.0 ? 1.0 : 0.0) :
     f_clamp01(1.0 - (ratio - 1.0) / (maxRatio - 1.0))

f_doubleQuality(
     float extremumDifference,
     float allowedDifference,
     float depth,
     float minimumDepth,
     float timeRatio,
     float maximumTimeRatio,
```

```
     float contextMove,
     float minimumContextMove,
     float span,
     float maximumSpan,
     float dominanceScore) =>
    float equalityScore =
        allowedDifference > syminfo.mintick ?
        f_clamp01(1.0 - extremumDifference / allowedDifference) :
        0.0

    float depthScore =
        minimumDepth > syminfo.mintick ?
        f_clamp01(depth / (minimumDepth * 1.8)) :
        0.0

    float contextScore =
        minimumContextMove > syminfo.mintick ?
        f_clamp01(contextMove / (minimumContextMove * 1.8)) :
        1.0

    float spanScore =
        maximumSpan > 0.0 ?
        f_clamp01(span / maximumSpan) :
        0.0

    math.min(
        100.0,
        equalityScore * 25.0 +
        depthScore * 25.0 +
        f_timeQuality(timeRatio, maximumTimeRatio) * 15.0 +
        contextScore * 15.0 +
        f_clamp01(dominanceScore) * 10.0 +
        spanScore * 10.0)

f_headShouldersQuality(
     float shoulderDifference,
     float allowedShoulderDifference,
     float headProminence,
     float headHeight,
     float minimumHeadDominanceRatio,
     float necklineSlopeAtr,
     float maximumNecklineSlopeAtr,
     float timeRatio,
     float maximumTimeRatio,
     float shoulderDepth,
     float minimumShoulderDepth,
     float contextMove,
     float minimumContextMove,
     float span,
     float maximumSpan,
     float dominanceScore) =>
    float shoulderScore =
        allowedShoulderDifference > syminfo.mintick ?
        f_clamp01(1.0 - shoulderDifference / allowedShoulderDifference) :
        0.0

    float headRatio =
        headHeight > syminfo.mintick ?
        headProminence / headHeight :
        0.0
```

```
    float headScore =
        minimumHeadDominanceRatio > 0.0 ?
        f_clamp01(headRatio / (minimumHeadDominanceRatio * 1.8)) :
        f_clamp01(headRatio)

    float necklineScore =
        maximumNecklineSlopeAtr > 0.0 ?
        f_clamp01(1.0 - necklineSlopeAtr / maximumNecklineSlopeAtr) :
        0.0

    float depthScore =
        minimumShoulderDepth > syminfo.mintick ?
        f_clamp01(shoulderDepth / (minimumShoulderDepth * 1.8)) :
        0.0

    float contextScore =
        minimumContextMove > syminfo.mintick ?
        f_clamp01(contextMove / (minimumContextMove * 1.8)) :
        1.0

    float spanScore =
        maximumSpan > 0.0 ?
        f_clamp01(span / maximumSpan) :
        0.0

    math.min(
        100.0,
        shoulderScore * 20.0 +
        headScore * 22.0 +
        necklineScore * 13.0 +
        f_timeQuality(timeRatio, maximumTimeRatio) * 14.0 +
        depthScore * 10.0 +
        contextScore * 10.0 +
        f_clamp01(dominanceScore) * 6.0 +
        spanScore * 5.0)

f_reversalRank(
     float quality,
     bool isMacro) =>
    float rank =
        nz(quality, 0.0) +
        (isMacro ? macroArbitrationBonus : 0.0)

    rank

f_reversalLifetime(
     int patternSpan,
     bool isMacro) =>
    float factor =
        isMacro ?
        macroReversalLifetimeFactor :
        microReversalLifetimeFactor

    int calculated =
        int(math.round(patternSpan * factor))

    int bounded =
        math.max(
            minAdaptiveReversalLifetime,
```

```
            math.min(
                maxAdaptiveReversalLifetime,
                calculated))

    adaptiveReversalLifetime ?
     bounded :
     reversalConfirmBars

f_newPatternLine(
     int x1,
     float y1,
     int x2,
     float y2,
     color patternColor,
     string patternStyle) =>
    string selectedStyle =
        patternStyle == "DASHED" ?
        line.style_dashed :
        line.style_dotted

    line.new(
        x1=x1,
        y1=y1,
        x2=x2,
        y2=y2,
        xloc=xloc.bar_index,
        extend=extend.none,
        color=patternColor,
        style=selectedStyle,
        width=3)

f_clearPattern3(EvaPattern3 pattern) =>
    if not na(pattern.leg1)
        line.delete(pattern.leg1)
    if not na(pattern.leg2)
        line.delete(pattern.leg2)
    if not na(pattern.neck)
        line.delete(pattern.neck)

    pattern.leg1 := na
    pattern.leg2 := na
    pattern.neck := na
    true

f_clearPattern5(EvaPattern5 pattern) =>
    if not na(pattern.leg1)
        line.delete(pattern.leg1)
    if not na(pattern.leg2)
        line.delete(pattern.leg2)
    if not na(pattern.leg3)
        line.delete(pattern.leg3)
    if not na(pattern.leg4)
        line.delete(pattern.leg4)
    if not na(pattern.neck)
        line.delete(pattern.neck)

    pattern.leg1 := na
    pattern.leg2 := na
    pattern.leg3 := na
    pattern.leg4 := na
```

```
    pattern.neck := na
    true

f_keepPatternLine(
     int x1,
     float y1,
     int x2,
     float y2,
     color patternColor,
     string patternStyle) =>
    line patternLine = f_newPatternLine(
         x1,
         y1,
         x2,
         y2,
         patternColor,
         patternStyle)

    f_keepLine(patternLine)
    true

f_drawDoublePattern(
     EvaPattern3 pattern,
     bool isLong,
     float neckline,
     int confirmBar,
     float quality,
     bool isMacro) =>
    color patternColor =
         isLong ?
         doubleBottomColor :
         doubleTopColor

    if na(pattern.leg1)
        pattern.leg1 := f_newPatternLine(
             pattern.x1,
             pattern.y1,
             pattern.x2,
             pattern.y2,
             patternColor,
             "DASHED")
    else
        line.set_color(pattern.leg1, patternColor)

    if na(pattern.leg2)
        pattern.leg2 := f_newPatternLine(
             pattern.x2,
             pattern.y2,
             pattern.x3,
             pattern.y3,
             patternColor,
             "DASHED")
    else
        line.set_color(pattern.leg2, patternColor)

    if na(pattern.neck)
        pattern.neck := f_newPatternLine(
             pattern.x2,
             neckline,
             confirmBar,
```

```
            neckline,
            patternColor,
            "DASHED")
    else
        line.set_xy1(pattern.neck, pattern.x2, neckline)
        line.set_xy2(pattern.neck, confirmBar, neckline)
        line.set_color(pattern.neck, patternColor)

    line.set_style(pattern.leg1, line.style_dashed)
    line.set_style(pattern.leg2, line.style_dashed)
    line.set_style(pattern.neck, line.style_dashed)
    line.set_width(pattern.leg1, 3)
    line.set_width(pattern.leg2, 3)
    line.set_width(pattern.neck, 3)

    f_keepLine(pattern.leg1)
    f_keepLine(pattern.leg2)
    f_keepLine(pattern.neck)

    string patternName =
        isLong ?
        "ДВОЙНОЕ ДНО" :
        "ДВОЙНАЯ ВЕРШИНА"

    label patternLabel = label.new(
        x=confirmBar,
        y=isLong ? low : high,
        text=patternName +
            "\n" +
            (isLong ? "ЛОНГ" : "ШОРТ") +
            " · КАЧ. " +
            str.tostring(quality, "#") +
            "%" +
            "\n" +
            (isMacro ? "КРУПНАЯ" : "ЛОКАЛЬНАЯ") +
            " · " +
            str.tostring(pattern.x3 - pattern.x1) +
            " баров",
        xloc=xloc.bar_index,
        yloc=isLong ? yloc.belowbar : yloc.abovebar,
        style=isLong ? label.style_label_up : label.style_label_down,
        color=patternColor,
        textcolor=color.white,
        size=size.small)

    f_keepLabel(patternLabel)
    f_trimConfirmed()

    pattern.leg1 := na
    pattern.leg2 := na
    pattern.neck := na
    true

f_drawHeadShoulders(
     EvaPattern5 pattern,
     bool isLong,
     int confirmBar,
     float confirmNeckline,
     float quality,
     bool isMacro) =>
```

```
    color patternColor =
         isLong ?
         inverseHeadShouldersColor :
         headShouldersColor

    if na(pattern.leg1)
        pattern.leg1 := f_newPatternLine(pattern.x1, pattern.y1, pattern.x2, pattern.y2, patternColor,
"DOTTED")
    else
        line.set_color(pattern.leg1, patternColor)

    if na(pattern.leg2)
        pattern.leg2 := f_newPatternLine(pattern.x2, pattern.y2, pattern.x3, pattern.y3, patternColor,
"DOTTED")
    else
        line.set_color(pattern.leg2, patternColor)

    if na(pattern.leg3)
        pattern.leg3 := f_newPatternLine(pattern.x3, pattern.y3, pattern.x4, pattern.y4, patternColor,
"DOTTED")
    else
        line.set_color(pattern.leg3, patternColor)

    if na(pattern.leg4)
        pattern.leg4 := f_newPatternLine(pattern.x4, pattern.y4, pattern.x5, pattern.y5, patternColor,
"DOTTED")
    else
        line.set_color(pattern.leg4, patternColor)

    if na(pattern.neck)
        pattern.neck := f_newPatternLine(pattern.x2, pattern.y2, confirmBar, confirmNeckline,
patternColor, "DOTTED")
    else
        line.set_xy1(pattern.neck, pattern.x2, pattern.y2)
        line.set_xy2(pattern.neck, confirmBar, confirmNeckline)
        line.set_color(pattern.neck, patternColor)

    line.set_style(pattern.leg1, line.style_dotted)
    line.set_style(pattern.leg2, line.style_dotted)
    line.set_style(pattern.leg3, line.style_dotted)
    line.set_style(pattern.leg4, line.style_dotted)
    line.set_style(pattern.neck, line.style_dotted)

    line.set_width(pattern.leg1, 3)
    line.set_width(pattern.leg2, 3)
    line.set_width(pattern.leg3, 3)
    line.set_width(pattern.leg4, 3)
    line.set_width(pattern.neck, 3)

    f_keepLine(pattern.leg1)
    f_keepLine(pattern.leg2)
    f_keepLine(pattern.leg3)
    f_keepLine(pattern.leg4)
    f_keepLine(pattern.neck)

    string patternName =
         isLong ?
         "ПЕРЕВЁРНУТЫЕ ГОЛОВА И ПЛЕЧИ" :
         "ГОЛОВА И ПЛЕЧИ"
```

```
    label patternLabel = label.new(
        x=confirmBar,
        y=isLong ? low : high,
        text=patternName +
            "\n" +
            (isLong ? "ЛОНГ" : "ШОРТ") +
            " · КАЧ. " +
            str.tostring(quality, "#") +
            "%" +
            "\n" +
            (isMacro ? "КРУПНАЯ" : "ЛОКАЛЬНАЯ") +
            " · " +
            str.tostring(pattern.x5 - pattern.x1) +
            " баров",
        xloc=xloc.bar_index,
        yloc=isLong ? yloc.belowbar : yloc.abovebar,
        style=isLong ? label.style_label_up : label.style_label_down,
        color=patternColor,
        textcolor=color.white,
        size=size.small)

    f_keepLabel(patternLabel)
    f_trimConfirmed()

    pattern.leg1 := na
    pattern.leg2 := na
    pattern.leg3 := na
    pattern.leg4 := na
    pattern.neck := na
    true

float reversalPivotHigh =
     ta.pivothigh(
          high,
          reversalPivotLeft,
          reversalPivotRight)

float reversalPivotLow =
     ta.pivotlow(
          low,
          reversalPivotLeft,
          reversalPivotRight)

bool newReversalPivot = false

if not na(reversalPivotHigh)
    bool highChanged = f_addReversalPivot(
        1,
        bar_index - reversalPivotRight,
        reversalPivotHigh)

    newReversalPivot := newReversalPivot or highChanged

if not na(reversalPivotLow)
    bool lowChanged = f_addReversalPivot(
        -1,
        bar_index - reversalPivotRight,
        reversalPivotLow)

    newReversalPivot := newReversalPivot or lowChanged
```

```
float reversalAtr =
    nz(
        atr[reversalPivotRight],
        atr)

float macroReversalPivotHigh =
    enableMacroReversals ?
    ta.pivothigh(
        high,
        macroReversalPivotLeft,
        macroReversalPivotRight) :
    na

float macroReversalPivotLow =
    enableMacroReversals ?
    ta.pivotlow(
        low,
        macroReversalPivotLeft,
        macroReversalPivotRight) :
    na

bool newMacroReversalPivot = false

if enableMacroReversals and not na(macroReversalPivotHigh)
   bool highChanged = f_addMacroReversalPivot(
       1,
       bar_index - macroReversalPivotRight,
       macroReversalPivotHigh)

   newMacroReversalPivot := newMacroReversalPivot or highChanged

if enableMacroReversals and not na(macroReversalPivotLow)
   bool lowChanged = f_addMacroReversalPivot(
       -1,
       bar_index - macroReversalPivotRight,
       macroReversalPivotLow)

   newMacroReversalPivot := newMacroReversalPivot or lowChanged

float macroReversalAtr =
    nz(
        atr[macroReversalPivotRight],
        atr)

if newReversalPivot
   int pivotCount = array.size(reversalTypes)

   if pivotCount >= 3
      int i1 = pivotCount - 3
      int i2 = pivotCount - 2
      int i3 = pivotCount - 1

      int t1 = array.get(reversalTypes, i1)
      int t2 = array.get(reversalTypes, i2)
      int t3 = array.get(reversalTypes, i3)

      int x1 = array.get(reversalBars, i1)
      int x2 = array.get(reversalBars, i2)
      int x3 = array.get(reversalBars, i3)
```

```
float y1 = array.get(reversalPrices, i1)
float y2 = array.get(reversalPrices, i2)
float y3 = array.get(reversalPrices, i3)

bool spacing3 =
    x2 - x1 >= minPivotGap and
    x3 - x2 >= minPivotGap and
    x3 - x1 >= minReversalSpan and
    x3 - x1 <= maxReversalSpan

float doubleLeftSpan = x2 - x1
float doubleRightSpan = x3 - x2

float doubleTimeRatio =
    math.max(doubleLeftSpan, doubleRightSpan) /
    math.max(math.min(doubleLeftSpan, doubleRightSpan), 1.0)

bool doubleTimeSymmetryOk =
    doubleTimeRatio <= maxDoubleTimeRatio

bool hasDoubleContext = pivotCount >= 4

int contextType3 =
    hasDoubleContext ?
    array.get(reversalTypes, pivotCount - 4) :
    0

float contextPrice3 =
    hasDoubleContext ?
    array.get(reversalPrices, pivotCount - 4) :
    na

float doubleBottomContextMove =
    hasDoubleContext and contextType3 == 1 ?
    contextPrice3 - y1 :
    0.0

float doubleTopContextMove =
    hasDoubleContext and contextType3 == -1 ?
    y1 - contextPrice3 :
    0.0

bool doubleBottomContextOk =
    not requireReversalTrendContext or
    doubleBottomContextMove >=
    minApproachMoveAtr * reversalAtr

bool doubleTopContextOk =
    not requireReversalTrendContext or
    doubleTopContextMove >=
    minApproachMoveAtr * reversalAtr

float microDoubleTolerance =
    doubleToleranceAtr * reversalAtr

float doubleBottomDepth =
    y2 - math.max(y1, y3)

float doubleTopDepth =
```

```
    math.min(y1, y3) - y2

float doubleBottomQuality =
    f_doubleQuality(
        math.abs(y1 - y3),
        microDoubleTolerance,
        doubleBottomDepth,
        minDoubleDepthAtr * reversalAtr,
        doubleTimeRatio,
        maxDoubleTimeRatio,
        requireReversalTrendContext ?
            doubleBottomContextMove :
            minApproachMoveAtr * reversalAtr,
        minApproachMoveAtr * reversalAtr,
        x3 - x1,
        maxReversalSpan,
        1.0)

float doubleTopQuality =
    f_doubleQuality(
        math.abs(y1 - y3),
        microDoubleTolerance,
        doubleTopDepth,
        minDoubleDepthAtr * reversalAtr,
        doubleTimeRatio,
        maxDoubleTimeRatio,
        requireReversalTrendContext ?
            doubleTopContextMove :
            minApproachMoveAtr * reversalAtr,
        minApproachMoveAtr * reversalAtr,
        x3 - x1,
        maxReversalSpan,
        1.0)

bool doubleBottomGeometry =
    showDoublePatterns and
    t1 == -1 and
    t2 == 1 and
    t3 == -1 and
    spacing3 and
    doubleTimeSymmetryOk and
    doubleBottomContextOk and
    math.abs(y1 - y3) <= microDoubleTolerance and
    doubleBottomDepth >= minDoubleDepthAtr * reversalAtr and
    doubleBottomQuality >= minFormingReversalQuality and
    (na(lastDoubleBottomPivot) or x3 != lastDoubleBottomPivot)

bool acceptDoubleBottom =
    doubleBottomGeometry and
    (
        not pendingDoubleBottom.active or
        f_reversalRank(doubleBottomQuality, false) >
        f_reversalRank(
            pendingDoubleBottomQuality,
            pendingDoubleBottomIsMacro) +
        reversalReplaceMargin or
        (
            x3 == pendingDoubleBottom.x3 and
            doubleBottomQuality >
            nz(pendingDoubleBottomQuality, 0.0)
```

```
        )
    )

if acceptDoubleBottom
    f_clearPattern3(pendingDoubleBottom)

    pendingDoubleBottom.active := true
    pendingDoubleBottomIsMacro := false
    pendingDoubleBottomQuality := doubleBottomQuality
    pendingDoubleBottom.x1 := x1
    pendingDoubleBottom.y1 := y1
    pendingDoubleBottom.x2 := x2
    pendingDoubleBottom.y2 := y2
    pendingDoubleBottom.x3 := x3
    pendingDoubleBottom.y3 := y3

    if showFormingReversals
        color candidateColor =
            color.new(
                doubleBottomColor,
                formingPatternTransparency)

        pendingDoubleBottom.leg1 := f_newPatternLine(
            x1, y1, x2, y2, candidateColor, "DASHED")

        pendingDoubleBottom.leg2 := f_newPatternLine(
            x2, y2, x3, y3, candidateColor, "DASHED")

        pendingDoubleBottom.neck := f_newPatternLine(
            x2, y2, bar_index, y2, candidateColor, "DASHED")

bool doubleTopGeometry =
    showDoublePatterns and
    t1 == 1 and
    t2 == -1 and
    t3 == 1 and
    spacing3 and
    doubleTimeSymmetryOk and
    doubleTopContextOk and
    math.abs(y1 - y3) <= microDoubleTolerance and
    doubleTopDepth >= minDoubleDepthAtr * reversalAtr and
    doubleTopQuality >= minFormingReversalQuality and
    (na(lastDoubleTopPivot) or x3 != lastDoubleTopPivot)

bool acceptDoubleTop =
    doubleTopGeometry and
    (
        not pendingDoubleTop.active or
        f_reversalRank(doubleTopQuality, false) >
        f_reversalRank(
            pendingDoubleTopQuality,
            pendingDoubleTopIsMacro) +
        reversalReplaceMargin or
        (
            x3 == pendingDoubleTop.x3 and
            doubleTopQuality >
            nz(pendingDoubleTopQuality, 0.0)
        )
    )
```

```
    if acceptDoubleTop
        f_clearPattern3(pendingDoubleTop)

        pendingDoubleTop.active := true
        pendingDoubleTopIsMacro := false
        pendingDoubleTopQuality := doubleTopQuality
        pendingDoubleTop.x1 := x1
        pendingDoubleTop.y1 := y1
        pendingDoubleTop.x2 := x2
        pendingDoubleTop.y2 := y2
        pendingDoubleTop.x3 := x3
        pendingDoubleTop.y3 := y3

        if showFormingReversals
            color candidateColor =
                color.new(
                    doubleTopColor,
                    formingPatternTransparency)

            pendingDoubleTop.leg1 := f_newPatternLine(
                x1, y1, x2, y2, candidateColor, "DASHED")

            pendingDoubleTop.leg2 := f_newPatternLine(
                x2, y2, x3, y3, candidateColor, "DASHED")

            pendingDoubleTop.neck := f_newPatternLine(
                x2, y2, bar_index, y2, candidateColor, "DASHED")

if pivotCount >= 5
    int i1 = pivotCount - 5
    int i2 = pivotCount - 4
    int i3 = pivotCount - 3
    int i4 = pivotCount - 2
    int i5 = pivotCount - 1

    int t1 = array.get(reversalTypes, i1)
    int t2 = array.get(reversalTypes, i2)
    int t3 = array.get(reversalTypes, i3)
    int t4 = array.get(reversalTypes, i4)
    int t5 = array.get(reversalTypes, i5)

    int x1 = array.get(reversalBars, i1)
    int x2 = array.get(reversalBars, i2)
    int x3 = array.get(reversalBars, i3)
    int x4 = array.get(reversalBars, i4)
    int x5 = array.get(reversalBars, i5)

    float y1 = array.get(reversalPrices, i1)
    float y2 = array.get(reversalPrices, i2)
    float y3 = array.get(reversalPrices, i3)
    float y4 = array.get(reversalPrices, i4)
    float y5 = array.get(reversalPrices, i5)

    bool spacing5 =
        x2 - x1 >= minPivotGap and
        x3 - x2 >= minPivotGap and
        x4 - x3 >= minPivotGap and
        x5 - x4 >= minPivotGap and
        x5 - x1 >= minReversalSpan and
        x5 - x1 <= maxReversalSpan
```

```
float necklineSlope =
    (y4 - y2) /
    math.max(x4 - x2, 1)

float necklineSlopeAtr =
    math.abs(necklineSlope) /
    math.max(reversalAtr, syminfo.mintick)

bool necklineSlopeOk =
    necklineSlopeAtr <= maxNecklineSlopeAtr

float leftShoulderSpan = x3 - x1
float rightShoulderSpan = x5 - x3

float shoulderTimeRatio =
    math.max(leftShoulderSpan, rightShoulderSpan) /
    math.max(math.min(leftShoulderSpan, rightShoulderSpan), 1.0)

bool shoulderTimeSymmetryOk =
    shoulderTimeRatio <= maxShoulderTimeRatio

bool hasShoulderContext = pivotCount >= 6

int contextType5 =
    hasShoulderContext ?
    array.get(reversalTypes, pivotCount - 6) :
    0

float contextPrice5 =
    hasShoulderContext ?
    array.get(reversalPrices, pivotCount - 6) :
    na

float headShouldersContextMove =
    hasShoulderContext and contextType5 == -1 ?
    y1 - contextPrice5 :
    0.0

float inverseContextMove =
    hasShoulderContext and contextType5 == 1 ?
    contextPrice5 - y1 :
    0.0

bool headShouldersContextOk =
    not requireReversalTrendContext or
    headShouldersContextMove >=
    minApproachMoveAtr * reversalAtr

bool inverseContextOk =
    not requireReversalTrendContext or
    inverseContextMove >=
    minApproachMoveAtr * reversalAtr

float shoulderDepth =
    math.min(y1, y5) - math.max(y2, y4)

float inverseShoulderDepth =
    math.min(y2, y4) - math.max(y1, y5)
```

```
float necklineAverage = (y2 + y4) * 0.5
float headHeightTop = y3 - necklineAverage
float headHeightInverse = necklineAverage - y3
float headProminenceTop = y3 - math.max(y1, y5)
float headProminenceInverse = math.min(y1, y5) - y3
float microShoulderTolerance = shoulderToleranceAtr * reversalAtr

float headShouldersQuality =
    f_headShouldersQuality(
        math.abs(y1 - y5),
        microShoulderTolerance,
        headProminenceTop,
        headHeightTop,
        minHeadHeightAtr /
        math.max(minHeadHeightAtr + 1.0, 1.0),
        necklineSlopeAtr,
        maxNecklineSlopeAtr,
        shoulderTimeRatio,
        maxShoulderTimeRatio,
        shoulderDepth,
        minShoulderDepthAtr * reversalAtr,
        requireReversalTrendContext ?
            headShouldersContextMove :
            minApproachMoveAtr * reversalAtr,
        minApproachMoveAtr * reversalAtr,
        x5 - x1,
        maxReversalSpan,
        1.0)

float inverseHeadShouldersQuality =
    f_headShouldersQuality(
        math.abs(y1 - y5),
        microShoulderTolerance,
        headProminenceInverse,
        headHeightInverse,
        minHeadHeightAtr /
        math.max(minHeadHeightAtr + 1.0, 1.0),
        necklineSlopeAtr,
        maxNecklineSlopeAtr,
        shoulderTimeRatio,
        maxShoulderTimeRatio,
        inverseShoulderDepth,
        minShoulderDepthAtr * reversalAtr,
        requireReversalTrendContext ?
            inverseContextMove :
            minApproachMoveAtr * reversalAtr,
        minApproachMoveAtr * reversalAtr,
        x5 - x1,
        maxReversalSpan,
        1.0)

bool headShouldersGeometry =
    showHeadShoulders and
    t1 == 1 and
    t2 == -1 and
    t3 == 1 and
    t4 == -1 and
    t5 == 1 and
    spacing5 and
    necklineSlopeOk and
```

```
    shoulderTimeSymmetryOk and
    headShouldersContextOk and
    shoulderDepth >= minShoulderDepthAtr * reversalAtr and
    math.abs(y1 - y5) <= microShoulderTolerance and
    headProminenceTop >= minHeadHeightAtr * reversalAtr and
    headShouldersQuality >= minFormingReversalQuality and
    (na(lastHeadShouldersPivot) or x5 != lastHeadShouldersPivot)

bool acceptHeadShoulders =
    headShouldersGeometry and
    (
        not pendingHeadShoulders.active or
        f_reversalRank(headShouldersQuality, false) >
        f_reversalRank(
            pendingHeadShouldersQuality,
            pendingHeadShouldersIsMacro) +
        reversalReplaceMargin or
        (
            x5 == pendingHeadShoulders.x5 and
            headShouldersQuality >
            nz(pendingHeadShouldersQuality, 0.0)
        )
    )

if acceptHeadShoulders
    f_clearPattern5(pendingHeadShoulders)

    pendingHeadShoulders.active := true
    pendingHeadShouldersIsMacro := false
    pendingHeadShouldersQuality := headShouldersQuality
    pendingHeadShoulders.x1 := x1
    pendingHeadShoulders.y1 := y1
    pendingHeadShoulders.x2 := x2
    pendingHeadShoulders.y2 := y2
    pendingHeadShoulders.x3 := x3
    pendingHeadShoulders.y3 := y3
    pendingHeadShoulders.x4 := x4
    pendingHeadShoulders.y4 := y4
    pendingHeadShoulders.x5 := x5
    pendingHeadShoulders.y5 := y5

    if showFormingReversals
        color candidateColor =
            color.new(
                headShouldersColor,
                formingPatternTransparency)

        pendingHeadShoulders.leg1 := f_newPatternLine(
            x1, y1, x2, y2, candidateColor, "DOTTED")
        pendingHeadShoulders.leg2 := f_newPatternLine(
            x2, y2, x3, y3, candidateColor, "DOTTED")
        pendingHeadShoulders.leg3 := f_newPatternLine(
            x3, y3, x4, y4, candidateColor, "DOTTED")
        pendingHeadShoulders.leg4 := f_newPatternLine(
            x4, y4, x5, y5, candidateColor, "DOTTED")

        float candidateNeckline =
            f_necklineValue(x2, y2, x4, y4, bar_index)

        pendingHeadShoulders.neck := f_newPatternLine(
```

```
            x2,
            y2,
            bar_index,
            candidateNeckline,
            candidateColor,
            "DOTTED")

bool inverseGeometry =
    showHeadShoulders and
    t1 == -1 and
    t2 == 1 and
    t3 == -1 and
    t4 == 1 and
    t5 == -1 and
    spacing5 and
    necklineSlopeOk and
    shoulderTimeSymmetryOk and
    inverseContextOk and
    inverseShoulderDepth >= minShoulderDepthAtr * reversalAtr and
    math.abs(y1 - y5) <= microShoulderTolerance and
    headProminenceInverse >= minHeadHeightAtr * reversalAtr and
    inverseHeadShouldersQuality >= minFormingReversalQuality and
    (na(lastInverseHeadShouldersPivot) or
     x5 != lastInverseHeadShouldersPivot)

bool acceptInverse =
    inverseGeometry and
    (
        not pendingInverseHeadShoulders.active or
        f_reversalRank(inverseHeadShouldersQuality, false) >
        f_reversalRank(
            pendingInverseHeadShouldersQuality,
            pendingInverseHeadShouldersIsMacro) +
        reversalReplaceMargin or
        (
            x5 == pendingInverseHeadShoulders.x5 and
            inverseHeadShouldersQuality >
            nz(pendingInverseHeadShouldersQuality, 0.0)
        )
    )

if acceptInverse
    f_clearPattern5(pendingInverseHeadShoulders)

    pendingInverseHeadShoulders.active := true
    pendingInverseHeadShouldersIsMacro := false
    pendingInverseHeadShouldersQuality := inverseHeadShouldersQuality
    pendingInverseHeadShoulders.x1 := x1
    pendingInverseHeadShoulders.y1 := y1
    pendingInverseHeadShoulders.x2 := x2
    pendingInverseHeadShoulders.y2 := y2
    pendingInverseHeadShoulders.x3 := x3
    pendingInverseHeadShoulders.y3 := y3
    pendingInverseHeadShoulders.x4 := x4
    pendingInverseHeadShoulders.y4 := y4
    pendingInverseHeadShoulders.x5 := x5
    pendingInverseHeadShoulders.y5 := y5

    if showFormingReversals
        color candidateColor =
```

```
                color.new(
                    inverseHeadShouldersColor,
                    formingPatternTransparency)

            pendingInverseHeadShoulders.leg1 := f_newPatternLine(
                x1, y1, x2, y2, candidateColor, "DOTTED")
            pendingInverseHeadShoulders.leg2 := f_newPatternLine(
                x2, y2, x3, y3, candidateColor, "DOTTED")
            pendingInverseHeadShoulders.leg3 := f_newPatternLine(
                x3, y3, x4, y4, candidateColor, "DOTTED")
            pendingInverseHeadShoulders.leg4 := f_newPatternLine(
                x4, y4, x5, y5, candidateColor, "DOTTED")

            float candidateNeckline =
                f_necklineValue(x2, y2, x4, y4, bar_index)

            pendingInverseHeadShoulders.neck := f_newPatternLine(
                x2,
                y2,
                bar_index,
                candidateNeckline,
                candidateColor,
                "DOTTED")

if enableMacroReversals and newMacroReversalPivot
    int pivotCount = array.size(macroReversalTypes)
    int effectiveMacroLookback =
        enableMacroCombinationSearch ?
        macroCombinationLookback :
        5

    int effectiveMacroTail =
        enableMacroCombinationSearch ?
        macroMaxTailPivots :
        0

    int scanStart =
        math.max(
            0,
            pivotCount - effectiveMacroLookback)

    float bestDoubleBottomQuality = -1.0
    int bestDoubleBottomX1 = na
    int bestDoubleBottomX2 = na
    int bestDoubleBottomX3 = na
    float bestDoubleBottomY1 = na
    float bestDoubleBottomY2 = na
    float bestDoubleBottomY3 = na

    float bestDoubleTopQuality = -1.0
    int bestDoubleTopX1 = na
    int bestDoubleTopX2 = na
    int bestDoubleTopX3 = na
    float bestDoubleTopY1 = na
    float bestDoubleTopY2 = na
    float bestDoubleTopY3 = na

    float bestHeadShouldersQuality = -1.0
    int bestHeadShouldersX1 = na
    int bestHeadShouldersX2 = na
```

```
int bestHeadShouldersX3 = na
int bestHeadShouldersX4 = na
int bestHeadShouldersX5 = na
float bestHeadShouldersY1 = na
float bestHeadShouldersY2 = na
float bestHeadShouldersY3 = na
float bestHeadShouldersY4 = na
float bestHeadShouldersY5 = na

float bestInverseQuality = -1.0
int bestInverseX1 = na
int bestInverseX2 = na
int bestInverseX3 = na
int bestInverseX4 = na
int bestInverseX5 = na
float bestInverseY1 = na
float bestInverseY2 = na
float bestInverseY3 = na
float bestInverseY4 = na
float bestInverseY5 = na

if pivotCount >= 3
    for dI1 = scanStart to pivotCount - 3
        for dI2 = dI1 + 1 to pivotCount - 2
            for dI3 = dI2 + 1 to pivotCount - 1
                int dT1 = array.get(macroReversalTypes, dI1)
                int dT2 = array.get(macroReversalTypes, dI2)
                int dT3 = array.get(macroReversalTypes, dI3)

                bool doubleTypeOk =
                    (dT1 == -1 and dT2 == 1 and dT3 == -1) or
                    (dT1 == 1 and dT2 == -1 and dT3 == 1)

                bool tailOk =
                    pivotCount - 1 - dI3 <= effectiveMacroTail

                if doubleTypeOk and tailOk
                    int dX1 = array.get(macroReversalBars, dI1)
                    int dX2 = array.get(macroReversalBars, dI2)
                    int dX3 = array.get(macroReversalBars, dI3)

                    float dY1 = array.get(macroReversalPrices, dI1)
                    float dY2 = array.get(macroReversalPrices, dI2)
                    float dY3 = array.get(macroReversalPrices, dI3)

                    bool spacing3 =
                        dX2 - dX1 >= minPivotGap and
                        dX3 - dX2 >= minPivotGap and
                        dX3 - dX1 >= macroMinReversalSpan and
                        dX3 - dX1 <= maxReversalSpan

                    float leftSpan = dX2 - dX1
                    float rightSpan = dX3 - dX2

                    float timeRatio =
                        math.max(leftSpan, rightSpan) /
                        math.max(math.min(leftSpan, rightSpan), 1.0)

                    bool timeOk =
                        timeRatio <= maxDoubleTimeRatio
```

```
bool hasContext = dI1 > 0

int contextType =
    hasContext ?
    array.get(macroReversalTypes, dI1 - 1) :
    0

float contextPrice =
    hasContext ?
    array.get(macroReversalPrices, dI1 - 1) :
    na

float intervalHigh = na
float intervalLow = na

for dK = dI1 to dI3
   int dKT = array.get(macroReversalTypes, dK)
   float dKP = array.get(macroReversalPrices, dK)

   if dKT == 1
      intervalHigh :=
         na(intervalHigh) ?
         dKP :
         math.max(intervalHigh, dKP)

   if dKT == -1
      intervalLow :=
         na(intervalLow) ?
         dKP :
         math.min(intervalLow, dKP)

if dT1 == -1
   float depth =
      dY2 - math.max(dY1, dY3)

   float tolerance =
      math.max(
         doubleToleranceAtr * macroReversalAtr,
         math.max(depth, 0.0) *
         macroDoubleToleranceRatio)

   float dominanceDeviation =
      math.max(
         nz(intervalHigh, dY2) - dY2,
         math.max(dY1, dY3) -
         nz(intervalLow, math.min(dY1, dY3)))

   float dominanceScore =
      tolerance > syminfo.mintick ?
      f_clamp01(
         1.0 -
         math.max(dominanceDeviation, 0.0) /
         tolerance) :
      0.0

   float contextMove =
      hasContext and contextType == 1 ?
      contextPrice - dY1 :
      0.0
```

```
    bool contextOk =
        not requireReversalTrendContext or
        contextMove >=
        minApproachMoveAtr * macroReversalAtr

    float quality =
        f_doubleQuality(
            math.abs(dY1 - dY3),
            tolerance,
            depth,
            minDoubleDepthAtr * macroReversalAtr,
            timeRatio,
            maxDoubleTimeRatio,
            requireReversalTrendContext ?
                contextMove :
                minApproachMoveAtr * macroReversalAtr,
            minApproachMoveAtr * macroReversalAtr,
            dX3 - dX1,
            maxReversalSpan,
            dominanceScore)

    bool geometry =
        showDoublePatterns and
        spacing3 and
        timeOk and
        contextOk and
        dominanceDeviation <= tolerance and
        math.abs(dY1 - dY3) <= tolerance and
        depth >= minDoubleDepthAtr * macroReversalAtr and
        quality >= minFormingReversalQuality and
        (na(lastDoubleBottomPivot) or
         dX3 != lastDoubleBottomPivot)

    if geometry and quality > bestDoubleBottomQuality
       bestDoubleBottomQuality := quality
       bestDoubleBottomX1 := dX1
       bestDoubleBottomX2 := dX2
       bestDoubleBottomX3 := dX3
       bestDoubleBottomY1 := dY1
       bestDoubleBottomY2 := dY2
       bestDoubleBottomY3 := dY3

if dT1 == 1
    float depth =
        math.min(dY1, dY3) - dY2

    float tolerance =
        math.max(
            doubleToleranceAtr * macroReversalAtr,
            math.max(depth, 0.0) *
            macroDoubleToleranceRatio)

    float dominanceDeviation =
        math.max(
            dY2 - nz(intervalLow, dY2),
            nz(intervalHigh, math.max(dY1, dY3)) -
            math.min(dY1, dY3))

    float dominanceScore =
```

```
                    tolerance > syminfo.mintick ?
                    f_clamp01(
                        1.0 -
                        math.max(dominanceDeviation, 0.0) /
                        tolerance) :
                    0.0

                float contextMove =
                    hasContext and contextType == -1 ?
                    dY1 - contextPrice :
                    0.0

                bool contextOk =
                    not requireReversalTrendContext or
                    contextMove >=
                    minApproachMoveAtr * macroReversalAtr

                float quality =
                    f_doubleQuality(
                        math.abs(dY1 - dY3),
                        tolerance,
                        depth,
                        minDoubleDepthAtr * macroReversalAtr,
                        timeRatio,
                        maxDoubleTimeRatio,
                        requireReversalTrendContext ?
                            contextMove :
                            minApproachMoveAtr * macroReversalAtr,
                        minApproachMoveAtr * macroReversalAtr,
                        dX3 - dX1,
                        maxReversalSpan,
                        dominanceScore)

                bool geometry =
                    showDoublePatterns and
                    spacing3 and
                    timeOk and
                    contextOk and
                    dominanceDeviation <= tolerance and
                    math.abs(dY1 - dY3) <= tolerance and
                    depth >= minDoubleDepthAtr * macroReversalAtr and
                    quality >= minFormingReversalQuality and
                    (na(lastDoubleTopPivot) or
                     dX3 != lastDoubleTopPivot)

                if geometry and quality > bestDoubleTopQuality
                    bestDoubleTopQuality := quality
                    bestDoubleTopX1 := dX1
                    bestDoubleTopX2 := dX2
                    bestDoubleTopX3 := dX3
                    bestDoubleTopY1 := dY1
                    bestDoubleTopY2 := dY2
                    bestDoubleTopY3 := dY3

if pivotCount >= 5
    for hI1 = scanStart to pivotCount - 5
        for hI2 = hI1 + 1 to pivotCount - 4
            for hI3 = hI2 + 1 to pivotCount - 3
                for hI4 = hI3 + 1 to pivotCount - 2
                    for hI5 = hI4 + 1 to pivotCount - 1
```

```
int hT1 = array.get(macroReversalTypes, hI1)
int hT2 = array.get(macroReversalTypes, hI2)
int hT3 = array.get(macroReversalTypes, hI3)
int hT4 = array.get(macroReversalTypes, hI4)
int hT5 = array.get(macroReversalTypes, hI5)

bool topTypeOk =
    hT1 == 1 and
    hT2 == -1 and
    hT3 == 1 and
    hT4 == -1 and
    hT5 == 1

bool inverseTypeOk =
    hT1 == -1 and
    hT2 == 1 and
    hT3 == -1 and
    hT4 == 1 and
    hT5 == -1

bool tailOk =
    pivotCount - 1 - hI5 <= effectiveMacroTail

if (topTypeOk or inverseTypeOk) and tailOk
    int hX1 = array.get(macroReversalBars, hI1)
    int hX2 = array.get(macroReversalBars, hI2)
    int hX3 = array.get(macroReversalBars, hI3)
    int hX4 = array.get(macroReversalBars, hI4)
    int hX5 = array.get(macroReversalBars, hI5)

    float hY1 = array.get(macroReversalPrices, hI1)
    float hY2 = array.get(macroReversalPrices, hI2)
    float hY3 = array.get(macroReversalPrices, hI3)
    float hY4 = array.get(macroReversalPrices, hI4)
    float hY5 = array.get(macroReversalPrices, hI5)

    bool spacing5 =
        hX2 - hX1 >= minPivotGap and
        hX3 - hX2 >= minPivotGap and
        hX4 - hX3 >= minPivotGap and
        hX5 - hX4 >= minPivotGap and
        hX5 - hX1 >= macroMinReversalSpan and
        hX5 - hX1 <= maxReversalSpan

    float necklineSlope =
        (hY4 - hY2) /
        math.max(hX4 - hX2, 1)

    float necklineSlopeAtr =
        math.abs(necklineSlope) /
        math.max(
            macroReversalAtr,
            syminfo.mintick)

    bool necklineOk =
        necklineSlopeAtr <= maxNecklineSlopeAtr

    float leftSpan = hX3 - hX1
    float rightSpan = hX5 - hX3
```

```
float timeRatio =
    math.max(leftSpan, rightSpan) /
    math.max(math.min(leftSpan, rightSpan), 1.0)

bool timeOk =
    timeRatio <= maxShoulderTimeRatio

bool hasContext = hI1 > 0

int contextType =
    hasContext ?
    array.get(macroReversalTypes, hI1 - 1) :
    0

float contextPrice =
    hasContext ?
    array.get(macroReversalPrices, hI1 - 1) :
    na

float intervalHigh = na
float intervalLow = na

for hK = hI1 to hI5
   int hKT = array.get(macroReversalTypes, hK)
   float hKP = array.get(macroReversalPrices, hK)

   if hKT == 1
      intervalHigh :=
          na(intervalHigh) ?
          hKP :
          math.max(intervalHigh, hKP)

   if hKT == -1
      intervalLow :=
          na(intervalLow) ?
          hKP :
          math.min(intervalLow, hKP)

float necklineAverage = (hY2 + hY4) * 0.5

if topTypeOk
   float headHeight = hY3 - necklineAverage
   float headProminence = hY3 - math.max(hY1, hY5)

   float shoulderTolerance =
       math.max(
           shoulderToleranceAtr * macroReversalAtr,
           math.max(headHeight, 0.0) *
           macroShoulderToleranceRatio)

   float shoulderDepth =
       math.min(hY1, hY5) -
       math.max(hY2, hY4)

   float dominanceDeviation =
       nz(intervalHigh, hY3) - hY3

   float dominanceTolerance =
       math.max(
           macroReversalAtr * 0.15,
```

```
        syminfo.mintick)

float dominanceScore =
    f_clamp01(
        1.0 -
        math.max(dominanceDeviation, 0.0) /
        dominanceTolerance)

float contextMove =
    hasContext and contextType == -1 ?
    hY1 - contextPrice :
    0.0

bool contextOk =
    not requireReversalTrendContext or
    contextMove >=
    minApproachMoveAtr * macroReversalAtr

bool headDominanceOk =
    headHeight > syminfo.mintick and
    headProminence / headHeight >=
    macroMinHeadDominanceRatio

float quality =
    f_headShouldersQuality(
        math.abs(hY1 - hY5),
        shoulderTolerance,
        headProminence,
        headHeight,
        macroMinHeadDominanceRatio,
        necklineSlopeAtr,
        maxNecklineSlopeAtr,
        timeRatio,
        maxShoulderTimeRatio,
        shoulderDepth,
        minShoulderDepthAtr * macroReversalAtr,
        requireReversalTrendContext ?
            contextMove :
            minApproachMoveAtr * macroReversalAtr,
        minApproachMoveAtr * macroReversalAtr,
        hX5 - hX1,
        maxReversalSpan,
        dominanceScore)

bool geometry =
    showHeadShoulders and
    spacing5 and
    necklineOk and
    timeOk and
    contextOk and
    shoulderDepth >=
    minShoulderDepthAtr * macroReversalAtr and
    headDominanceOk and
    dominanceDeviation <= dominanceTolerance and
    math.abs(hY1 - hY5) <= shoulderTolerance and
    headProminence >=
    minHeadHeightAtr * macroReversalAtr and
    quality >= minFormingReversalQuality and
    (na(lastHeadShouldersPivot) or
     hX5 != lastHeadShouldersPivot)
```

```
    if geometry and quality > bestHeadShouldersQuality
        bestHeadShouldersQuality := quality
        bestHeadShouldersX1 := hX1
        bestHeadShouldersX2 := hX2
        bestHeadShouldersX3 := hX3
        bestHeadShouldersX4 := hX4
        bestHeadShouldersX5 := hX5
        bestHeadShouldersY1 := hY1
        bestHeadShouldersY2 := hY2
        bestHeadShouldersY3 := hY3
        bestHeadShouldersY4 := hY4
        bestHeadShouldersY5 := hY5

if inverseTypeOk
    float headHeight = necklineAverage - hY3
    float headProminence = math.min(hY1, hY5) - hY3

    float shoulderTolerance =
        math.max(
            shoulderToleranceAtr * macroReversalAtr,
            math.max(headHeight, 0.0) *
            macroShoulderToleranceRatio)

    float shoulderDepth =
        math.min(hY2, hY4) -
        math.max(hY1, hY5)

    float dominanceDeviation =
        hY3 - nz(intervalLow, hY3)

    float dominanceTolerance =
        math.max(
            macroReversalAtr * 0.15,
            syminfo.mintick)

    float dominanceScore =
        f_clamp01(
            1.0 -
            math.max(dominanceDeviation, 0.0) /
            dominanceTolerance)

    float contextMove =
        hasContext and contextType == 1 ?
        contextPrice - hY1 :
        0.0

    bool contextOk =
        not requireReversalTrendContext or
        contextMove >=
        minApproachMoveAtr * macroReversalAtr

    bool headDominanceOk =
        headHeight > syminfo.mintick and
        headProminence / headHeight >=
        macroMinHeadDominanceRatio

    float quality =
        f_headShouldersQuality(
            math.abs(hY1 - hY5),
```

```
                    shoulderTolerance,
                    headProminence,
                    headHeight,
                    macroMinHeadDominanceRatio,
                    necklineSlopeAtr,
                    maxNecklineSlopeAtr,
                    timeRatio,
                    maxShoulderTimeRatio,
                    shoulderDepth,
                    minShoulderDepthAtr * macroReversalAtr,
                    requireReversalTrendContext ?
                        contextMove :
                        minApproachMoveAtr * macroReversalAtr,
                    minApproachMoveAtr * macroReversalAtr,
                    hX5 - hX1,
                    maxReversalSpan,
                    dominanceScore)

                bool geometry =
                    showHeadShoulders and
                    spacing5 and
                    necklineOk and
                    timeOk and
                    contextOk and
                    shoulderDepth >=
                    minShoulderDepthAtr * macroReversalAtr and
                    headDominanceOk and
                    dominanceDeviation <= dominanceTolerance and
                    math.abs(hY1 - hY5) <= shoulderTolerance and
                    headProminence >=
                    minHeadHeightAtr * macroReversalAtr and
                    quality >= minFormingReversalQuality and
                    (na(lastInverseHeadShouldersPivot) or
                     hX5 != lastInverseHeadShouldersPivot)

                if geometry and quality > bestInverseQuality
                    bestInverseQuality := quality
                    bestInverseX1 := hX1
                    bestInverseX2 := hX2
                    bestInverseX3 := hX3
                    bestInverseX4 := hX4
                    bestInverseX5 := hX5
                    bestInverseY1 := hY1
                    bestInverseY2 := hY2
                    bestInverseY3 := hY3
                    bestInverseY4 := hY4
                    bestInverseY5 := hY5

bool acceptBestDoubleBottom =
    bestDoubleBottomQuality >= minFormingReversalQuality and
    (
        not pendingDoubleBottom.active or
        f_reversalRank(bestDoubleBottomQuality, true) >
        f_reversalRank(
            pendingDoubleBottomQuality,
            pendingDoubleBottomIsMacro) +
        reversalReplaceMargin or
        (
            bestDoubleBottomX3 == pendingDoubleBottom.x3 and
            bestDoubleBottomQuality >
```

```
            nz(pendingDoubleBottomQuality, 0.0)
        )
    )

if acceptBestDoubleBottom
    f_clearPattern3(pendingDoubleBottom)

    pendingDoubleBottom.active := true
    pendingDoubleBottomIsMacro := true
    pendingDoubleBottomQuality := bestDoubleBottomQuality
    pendingDoubleBottom.x1 := bestDoubleBottomX1
    pendingDoubleBottom.y1 := bestDoubleBottomY1
    pendingDoubleBottom.x2 := bestDoubleBottomX2
    pendingDoubleBottom.y2 := bestDoubleBottomY2
    pendingDoubleBottom.x3 := bestDoubleBottomX3
    pendingDoubleBottom.y3 := bestDoubleBottomY3

    if showFormingReversals
        color candidateColor =
            color.new(
                doubleBottomColor,
                formingPatternTransparency)

        pendingDoubleBottom.leg1 := f_newPatternLine(
            bestDoubleBottomX1,
            bestDoubleBottomY1,
            bestDoubleBottomX2,
            bestDoubleBottomY2,
            candidateColor,
            "DASHED")

        pendingDoubleBottom.leg2 := f_newPatternLine(
            bestDoubleBottomX2,
            bestDoubleBottomY2,
            bestDoubleBottomX3,
            bestDoubleBottomY3,
            candidateColor,
            "DASHED")

        pendingDoubleBottom.neck := f_newPatternLine(
            bestDoubleBottomX2,
            bestDoubleBottomY2,
            bar_index,
            bestDoubleBottomY2,
            candidateColor,
            "DASHED")

bool acceptBestDoubleTop =
    bestDoubleTopQuality >= minFormingReversalQuality and
    (
        not pendingDoubleTop.active or
        f_reversalRank(bestDoubleTopQuality, true) >
        f_reversalRank(
            pendingDoubleTopQuality,
            pendingDoubleTopIsMacro) +
        reversalReplaceMargin or
        (
            bestDoubleTopX3 == pendingDoubleTop.x3 and
            bestDoubleTopQuality >
            nz(pendingDoubleTopQuality, 0.0)
```

```
        )
    )

if acceptBestDoubleTop
    f_clearPattern3(pendingDoubleTop)

    pendingDoubleTop.active := true
    pendingDoubleTopIsMacro := true
    pendingDoubleTopQuality := bestDoubleTopQuality
    pendingDoubleTop.x1 := bestDoubleTopX1
    pendingDoubleTop.y1 := bestDoubleTopY1
    pendingDoubleTop.x2 := bestDoubleTopX2
    pendingDoubleTop.y2 := bestDoubleTopY2
    pendingDoubleTop.x3 := bestDoubleTopX3
    pendingDoubleTop.y3 := bestDoubleTopY3

    if showFormingReversals
        color candidateColor =
            color.new(
                doubleTopColor,
                formingPatternTransparency)

        pendingDoubleTop.leg1 := f_newPatternLine(
            bestDoubleTopX1,
            bestDoubleTopY1,
            bestDoubleTopX2,
            bestDoubleTopY2,
            candidateColor,
            "DASHED")

        pendingDoubleTop.leg2 := f_newPatternLine(
            bestDoubleTopX2,
            bestDoubleTopY2,
            bestDoubleTopX3,
            bestDoubleTopY3,
            candidateColor,
            "DASHED")

        pendingDoubleTop.neck := f_newPatternLine(
            bestDoubleTopX2,
            bestDoubleTopY2,
            bar_index,
            bestDoubleTopY2,
            candidateColor,
            "DASHED")

bool acceptBestHeadShoulders =
    bestHeadShouldersQuality >= minFormingReversalQuality and
    (
        not pendingHeadShoulders.active or
        f_reversalRank(bestHeadShouldersQuality, true) >
        f_reversalRank(
            pendingHeadShouldersQuality,
            pendingHeadShouldersIsMacro) +
        reversalReplaceMargin or
        (
            bestHeadShouldersX5 == pendingHeadShoulders.x5 and
            bestHeadShouldersQuality >
            nz(pendingHeadShouldersQuality, 0.0)
        )
```

```
    )

if acceptBestHeadShoulders
    f_clearPattern5(pendingHeadShoulders)

    pendingHeadShoulders.active := true
    pendingHeadShouldersIsMacro := true
    pendingHeadShouldersQuality := bestHeadShouldersQuality
    pendingHeadShoulders.x1 := bestHeadShouldersX1
    pendingHeadShoulders.y1 := bestHeadShouldersY1
    pendingHeadShoulders.x2 := bestHeadShouldersX2
    pendingHeadShoulders.y2 := bestHeadShouldersY2
    pendingHeadShoulders.x3 := bestHeadShouldersX3
    pendingHeadShoulders.y3 := bestHeadShouldersY3
    pendingHeadShoulders.x4 := bestHeadShouldersX4
    pendingHeadShoulders.y4 := bestHeadShouldersY4
    pendingHeadShoulders.x5 := bestHeadShouldersX5
    pendingHeadShoulders.y5 := bestHeadShouldersY5

    if showFormingReversals
        color candidateColor =
            color.new(
                headShouldersColor,
                formingPatternTransparency)

        pendingHeadShoulders.leg1 := f_newPatternLine(
            bestHeadShouldersX1,
            bestHeadShouldersY1,
            bestHeadShouldersX2,
            bestHeadShouldersY2,
            candidateColor,
            "DOTTED")
        pendingHeadShoulders.leg2 := f_newPatternLine(
            bestHeadShouldersX2,
            bestHeadShouldersY2,
            bestHeadShouldersX3,
            bestHeadShouldersY3,
            candidateColor,
            "DOTTED")
        pendingHeadShoulders.leg3 := f_newPatternLine(
            bestHeadShouldersX3,
            bestHeadShouldersY3,
            bestHeadShouldersX4,
            bestHeadShouldersY4,
            candidateColor,
            "DOTTED")
        pendingHeadShoulders.leg4 := f_newPatternLine(
            bestHeadShouldersX4,
            bestHeadShouldersY4,
            bestHeadShouldersX5,
            bestHeadShouldersY5,
            candidateColor,
            "DOTTED")

        float candidateNeckline =
            f_necklineValue(
                bestHeadShouldersX2,
                bestHeadShouldersY2,
                bestHeadShouldersX4,
                bestHeadShouldersY4,
```

```
            bar_index)

        pendingHeadShoulders.neck := f_newPatternLine(
            bestHeadShouldersX2,
            bestHeadShouldersY2,
            bar_index,
            candidateNeckline,
            candidateColor,
            "DOTTED")

bool acceptBestInverse =
    bestInverseQuality >= minFormingReversalQuality and
    (
        not pendingInverseHeadShoulders.active or
        f_reversalRank(bestInverseQuality, true) >
        f_reversalRank(
            pendingInverseHeadShouldersQuality,
            pendingInverseHeadShouldersIsMacro) +
        reversalReplaceMargin or
        (
            bestInverseX5 == pendingInverseHeadShoulders.x5 and
            bestInverseQuality >
            nz(pendingInverseHeadShouldersQuality, 0.0)
        )
    )

if acceptBestInverse
    f_clearPattern5(pendingInverseHeadShoulders)

    pendingInverseHeadShoulders.active := true
    pendingInverseHeadShouldersIsMacro := true
    pendingInverseHeadShouldersQuality := bestInverseQuality
    pendingInverseHeadShoulders.x1 := bestInverseX1
    pendingInverseHeadShoulders.y1 := bestInverseY1
    pendingInverseHeadShoulders.x2 := bestInverseX2
    pendingInverseHeadShoulders.y2 := bestInverseY2
    pendingInverseHeadShoulders.x3 := bestInverseX3
    pendingInverseHeadShoulders.y3 := bestInverseY3
    pendingInverseHeadShoulders.x4 := bestInverseX4
    pendingInverseHeadShoulders.y4 := bestInverseY4
    pendingInverseHeadShoulders.x5 := bestInverseX5
    pendingInverseHeadShoulders.y5 := bestInverseY5

    if showFormingReversals
        color candidateColor =
            color.new(
                inverseHeadShouldersColor,
                formingPatternTransparency)

        pendingInverseHeadShoulders.leg1 := f_newPatternLine(
            bestInverseX1,
            bestInverseY1,
            bestInverseX2,
            bestInverseY2,
            candidateColor,
            "DOTTED")
        pendingInverseHeadShoulders.leg2 := f_newPatternLine(
            bestInverseX2,
            bestInverseY2,
            bestInverseX3,
```

```
            bestInverseY3,
            candidateColor,
            "DOTTED")
        pendingInverseHeadShoulders.leg3 := f_newPatternLine(
            bestInverseX3,
            bestInverseY3,
            bestInverseX4,
            bestInverseY4,
            candidateColor,
            "DOTTED")
        pendingInverseHeadShoulders.leg4 := f_newPatternLine(
            bestInverseX4,
            bestInverseY4,
            bestInverseX5,
            bestInverseY5,
            candidateColor,
            "DOTTED")

        float candidateNeckline =
            f_necklineValue(
                bestInverseX2,
                bestInverseY2,
                bestInverseX4,
                bestInverseY4,
                bar_index)

        pendingInverseHeadShoulders.neck := f_newPatternLine(
            bestInverseX2,
            bestInverseY2,
            bar_index,
            candidateNeckline,
            candidateColor,
            "DOTTED")

bool doubleBottomSignal = false
bool doubleTopSignal = false
bool headShouldersSignal = false
bool inverseHeadShouldersSignal = false

if pendingDoubleBottom.active
    if showFormingReversals and not na(pendingDoubleBottom.neck)
        line.set_xy2(
            pendingDoubleBottom.neck,
            bar_index,
            pendingDoubleBottom.y2)

    int activeLifetime =
        f_reversalLifetime(
            pendingDoubleBottom.x3 - pendingDoubleBottom.x1,
            pendingDoubleBottomIsMacro)

    bool expired =
        bar_index - pendingDoubleBottom.x3 >
        activeLifetime

    bool invalidated =
        low <
        math.min(
            pendingDoubleBottom.y1,
            pendingDoubleBottom.y3) -
```

```
        doubleToleranceAtr * atr

    float doubleBottomBreakLevel =
        pendingDoubleBottom.y2 +
        reversalBreakoutBufferAtr * atr

    bool breakoutBodyOk =
        close > open and
        math.abs(close - open) >=
        minReversalBreakoutBodyAtr * atr

    bool normalBreakoutConfirmed =
        barstate.isconfirmed and
        bar_index >
        pendingDoubleBottom.x3 + reversalPivotRight and
        pendingDoubleBottomQuality >= minConfirmedReversalQuality and
        close > doubleBottomBreakLevel and
        close[1] <= doubleBottomBreakLevel and
        breakoutBodyOk

    bool delayedMacroBreakoutConfirmed =
        macroAllowDelayedBreakout and
        pendingDoubleBottomIsMacro and
        barstate.isconfirmed and
        bar_index >=
        pendingDoubleBottom.x3 + macroReversalPivotRight and
        pendingDoubleBottomQuality >= minConfirmedReversalQuality and
        close >
        pendingDoubleBottom.y2 +
        macroLateConfirmBufferAtr * atr

    bool confirmed =
        normalBreakoutConfirmed or
        delayedMacroBreakoutConfirmed

    if confirmed
       doubleBottomSignal := true

       f_drawDoublePattern(
           pendingDoubleBottom,
           true,
           pendingDoubleBottom.y2,
           bar_index,
           pendingDoubleBottomQuality,
           pendingDoubleBottomIsMacro)

       lastDoubleBottomPivot := pendingDoubleBottom.x3
       pendingDoubleBottom.active := false
       pendingDoubleBottomIsMacro := false
       pendingDoubleBottomQuality := na
    else if expired or invalidated
       f_clearPattern3(pendingDoubleBottom)
       pendingDoubleBottom.active := false
       pendingDoubleBottomIsMacro := false
       pendingDoubleBottomQuality := na

if pendingDoubleTop.active
    if showFormingReversals and not na(pendingDoubleTop.neck)
       line.set_xy2(
           pendingDoubleTop.neck,
```

```
        bar_index,
        pendingDoubleTop.y2)

int activeLifetime =
    f_reversalLifetime(
         pendingDoubleTop.x3 - pendingDoubleTop.x1,
         pendingDoubleTopIsMacro)

bool expired =
    bar_index - pendingDoubleTop.x3 >
    activeLifetime

bool invalidated =
    high >
    math.max(
        pendingDoubleTop.y1,
        pendingDoubleTop.y3) +
    doubleToleranceAtr * atr

float doubleTopBreakLevel =
    pendingDoubleTop.y2 -
    reversalBreakoutBufferAtr * atr

bool breakoutBodyOk =
    close < open and
    math.abs(close - open) >=
    minReversalBreakoutBodyAtr * atr

bool normalBreakoutConfirmed =
    barstate.isconfirmed and
    bar_index >
    pendingDoubleTop.x3 + reversalPivotRight and
    pendingDoubleTopQuality >= minConfirmedReversalQuality and
    close < doubleTopBreakLevel and
    close[1] >= doubleTopBreakLevel and
    breakoutBodyOk

bool delayedMacroBreakoutConfirmed =
    macroAllowDelayedBreakout and
    pendingDoubleTopIsMacro and
    barstate.isconfirmed and
    bar_index >=
    pendingDoubleTop.x3 + macroReversalPivotRight and
    pendingDoubleTopQuality >= minConfirmedReversalQuality and
    close <
    pendingDoubleTop.y2 -
    macroLateConfirmBufferAtr * atr

bool confirmed =
    normalBreakoutConfirmed or
    delayedMacroBreakoutConfirmed

if confirmed
   doubleTopSignal := true

   f_drawDoublePattern(
       pendingDoubleTop,
       false,
       pendingDoubleTop.y2,
       bar_index,
```

```
            pendingDoubleTopQuality,
            pendingDoubleTopIsMacro)

        lastDoubleTopPivot := pendingDoubleTop.x3
        pendingDoubleTop.active := false
        pendingDoubleTopIsMacro := false
        pendingDoubleTopQuality := na
    else if expired or invalidated
        f_clearPattern3(pendingDoubleTop)
        pendingDoubleTop.active := false
        pendingDoubleTopIsMacro := false
        pendingDoubleTopQuality := na

if pendingHeadShoulders.active
    float necklineNow =
        f_necklineValue(
            pendingHeadShoulders.x2,
            pendingHeadShoulders.y2,
            pendingHeadShoulders.x4,
            pendingHeadShoulders.y4,
            bar_index)

    if showFormingReversals and not na(pendingHeadShoulders.neck)
        line.set_xy2(
            pendingHeadShoulders.neck,
            bar_index,
            necklineNow)

    int activeLifetime =
        f_reversalLifetime(
            pendingHeadShoulders.x5 - pendingHeadShoulders.x1,
            pendingHeadShouldersIsMacro)

    bool expired =
        bar_index - pendingHeadShoulders.x5 >
        activeLifetime

    bool invalidated =
        high >
        pendingHeadShoulders.y3 +
        shoulderToleranceAtr * atr

    float previousNeckline =
        f_necklineValue(
            pendingHeadShoulders.x2,
            pendingHeadShoulders.y2,
            pendingHeadShoulders.x4,
            pendingHeadShoulders.y4,
            bar_index - 1)

    float headShouldersBreakLevel =
        necklineNow -
        reversalBreakoutBufferAtr * atr

    float previousHeadShouldersBreakLevel =
        previousNeckline -
        reversalBreakoutBufferAtr * atr[1]

    bool breakoutBodyOk =
        close < open and
```

```
        math.abs(close - open) >=
        minReversalBreakoutBodyAtr * atr

    bool normalBreakoutConfirmed =
        barstate.isconfirmed and
        bar_index >
        pendingHeadShoulders.x5 + reversalPivotRight and
        pendingHeadShouldersQuality >= minConfirmedReversalQuality and
        close < headShouldersBreakLevel and
        close[1] >= previousHeadShouldersBreakLevel and
        breakoutBodyOk

    bool delayedMacroBreakoutConfirmed =
        macroAllowDelayedBreakout and
        pendingHeadShouldersIsMacro and
        barstate.isconfirmed and
        bar_index >=
        pendingHeadShoulders.x5 + macroReversalPivotRight and
        pendingHeadShouldersQuality >= minConfirmedReversalQuality and
        close <
        necklineNow -
        macroLateConfirmBufferAtr * atr

    bool confirmed =
        normalBreakoutConfirmed or
        delayedMacroBreakoutConfirmed

    if confirmed
       headShouldersSignal := true

       f_drawHeadShoulders(
           pendingHeadShoulders,
           false,
           bar_index,
           necklineNow,
           pendingHeadShouldersQuality,
           pendingHeadShouldersIsMacro)

       lastHeadShouldersPivot := pendingHeadShoulders.x5
       pendingHeadShoulders.active := false
       pendingHeadShouldersIsMacro := false
       pendingHeadShouldersQuality := na
    else if expired or invalidated
       f_clearPattern5(pendingHeadShoulders)
       pendingHeadShoulders.active := false
       pendingHeadShouldersIsMacro := false
       pendingHeadShouldersQuality := na

if pendingInverseHeadShoulders.active
    float necklineNow =
        f_necklineValue(
            pendingInverseHeadShoulders.x2,
            pendingInverseHeadShoulders.y2,
            pendingInverseHeadShoulders.x4,
            pendingInverseHeadShoulders.y4,
            bar_index)

    if showFormingReversals and not na(pendingInverseHeadShoulders.neck)
       line.set_xy2(
           pendingInverseHeadShoulders.neck,
```

```
        bar_index,
        necklineNow)

int activeLifetime =
    f_reversalLifetime(
        pendingInverseHeadShoulders.x5 - pendingInverseHeadShoulders.x1,
        pendingInverseHeadShouldersIsMacro)

bool expired =
    bar_index - pendingInverseHeadShoulders.x5 >
    activeLifetime

bool invalidated =
    low <
    pendingInverseHeadShoulders.y3 -
    shoulderToleranceAtr * atr

float previousNeckline =
    f_necklineValue(
        pendingInverseHeadShoulders.x2,
        pendingInverseHeadShoulders.y2,
        pendingInverseHeadShoulders.x4,
        pendingInverseHeadShoulders.y4,
        bar_index - 1)

float inverseHeadShouldersBreakLevel =
    necklineNow +
    reversalBreakoutBufferAtr * atr

float previousInverseHeadShouldersBreakLevel =
    previousNeckline +
    reversalBreakoutBufferAtr * atr[1]

bool breakoutBodyOk =
    close > open and
    math.abs(close - open) >=
    minReversalBreakoutBodyAtr * atr

bool normalBreakoutConfirmed =
    barstate.isconfirmed and
    bar_index >
    pendingInverseHeadShoulders.x5 + reversalPivotRight and
    pendingInverseHeadShouldersQuality >= minConfirmedReversalQuality and
    close > inverseHeadShouldersBreakLevel and
    close[1] <= previousInverseHeadShouldersBreakLevel and
    breakoutBodyOk

bool delayedMacroBreakoutConfirmed =
    macroAllowDelayedBreakout and
    pendingInverseHeadShouldersIsMacro and
    barstate.isconfirmed and
    bar_index >=
    pendingInverseHeadShoulders.x5 + macroReversalPivotRight and
    pendingInverseHeadShouldersQuality >= minConfirmedReversalQuality and
    close >
    necklineNow +
    macroLateConfirmBufferAtr * atr

bool confirmed =
    normalBreakoutConfirmed or
```

```
        delayedMacroBreakoutConfirmed

    if confirmed
        inverseHeadShouldersSignal := true

        f_drawHeadShoulders(
            pendingInverseHeadShoulders,
            true,
            bar_index,
            necklineNow,
            pendingInverseHeadShouldersQuality,
            pendingInverseHeadShouldersIsMacro)

        lastInverseHeadShouldersPivot :=
            pendingInverseHeadShoulders.x5

        pendingInverseHeadShoulders.active := false
        pendingInverseHeadShouldersIsMacro := false
        pendingInverseHeadShouldersQuality := na
    else if expired or invalidated
        f_clearPattern5(pendingInverseHeadShoulders)
        pendingInverseHeadShoulders.active := false
        pendingInverseHeadShouldersIsMacro := false
        pendingInverseHeadShouldersQuality := na

//
------------------------------------------------------------------------------------------------------------------------
// Алерты
//
------------------------------------------------------------------------------------------------------------------------

alertcondition(
    confirmedLong and confirmedFlag,
    title="EVA · ЛОНГ Флаг",
    message="EVA Ai+: подтверждён ФЛАГ ЛОНГ · {{ticker}} · {{interval}}")

alertcondition(
    confirmedShort and confirmedFlag,
    title="EVA · ШОРТ Флаг",
    message="EVA Ai+: подтверждён ФЛАГ ШОРТ · {{ticker}} · {{interval}}")

alertcondition(
    confirmedLong and confirmedPennant,
    title="EVA · ЛОНГ Вымпел",
    message="EVA Ai+: подтверждён ВЫМПЕЛ ЛОНГ · {{ticker}} · {{interval}}")

alertcondition(
    confirmedShort and confirmedPennant,
    title="EVA · ШОРТ Вымпел",
    message="EVA Ai+: подтверждён ВЫМПЕЛ ШОРТ · {{ticker}} · {{interval}}")


alertcondition(
    doubleBottomSignal,
    title="EVA · ЛОНГ Двойное дно",
    message="EVA Ai+: подтверждено двойное дно ЛОНГ · {{ticker}} · {{interval}}")

alertcondition(
```

```
    doubleTopSignal,
    title="EVA · ШОРТ Двойная вершина",
    message="EVA Ai+: подтверждена двойная вершина ШОРТ · {{ticker}} · {{interval}}")

alertcondition(
    headShouldersSignal,
    title="EVA · ШОРТ Голова и плечи",
    message="EVA Ai+: подтверждены Голова и плечи ШОРТ · {{ticker}} · {{interval}}")

alertcondition(
    inverseHeadShouldersSignal,
    title="EVA · ЛОНГ Перевёрнутые ГиП",
    message="EVA Ai+: подтверждены перевёрнутые Голова и плечи ЛОНГ · {{ticker}} ·
{{interval}}")
```